

A Quantum Approach to Stochastic Optimization in Insurance Underwriting

Mitchell Bordelon¹, Maurice Garfinkel¹, Vivek Dixit², Thomas Whitehead¹, Jenny Holzbauer¹, Guillermo Mijares Vilarino², Alberto Maldonado Romo², Abhijit Mitra², Vaibhaw Kumar^{*2}, and Jean Utke^{†1}

¹Allstate

²IBM Quantum

June 17, 2026

Abstract

The presence of stochastic elements in combinatorial optimization problems makes them particularly challenging, as such problems quickly become intractable for classical computers even at relatively small sizes. In this work, we propose a novel quantum-classical hybrid scheme for solving a class of stochastic optimization problems known as chance-constrained knapsack problems, in which item weights follow probability distributions and constraints may be violated within a specified risk tolerance. Our method employs knapsack-specific QAOA-based circuits to generate samples which, when combined with a new self-consistent classical recovery scheme introduced in this work, produce high-quality solutions. Experiments carried out on IBM Heron processors, using circuits with depths up to 177 and comprising 3443 gates acting on as many as 150 qubits, yield solutions that indicate performance comparable to classical optimization schemes. The proposed quantum-classical scheme paves the way to tackling such problems, with the potential to outperform approaches that rely solely on classical computation.¹

1 Introduction

Intrinsic stochasticity in combinatorial optimization problems introduces uncertainty that substantially increases computational complexity and limits the effectiveness of deterministic solution methods. This uncertainty naturally arises

^{*}vaibhaw.kumar@ibm.com

[†]jutke@allstate.com

¹The source code will be made publicly available upon acceptance of the manuscript.

in applications including finance, logistics, insurance, and other risk-reward business problems, where fluctuations in demand, prices, or system disruptions require risk-based decision-making. In these settings, uncertainty is typically modeled through chance-constrained formulations, in which constraints are enforced only probabilistically and violations are allowed with a small probability ϵ that reflects a specified level of risk tolerance. Decision making under such risk-reward trade-offs amounts to balancing probabilistic feasibility against optimization objectives.

In particular, decision tasks in insurance underwriting naturally map to chance-constrained knapsack (CCKP) problems. By abstracting away operational domain-specific details, CCKP formulations capture the core insurance objective weighing expected return against acceptable risk, while preserving a defining feature of the problem: the stochastic nature of constraints. In real-world scenarios, this stochasticity stems from predictive models used to assess risk and estimate uncertainty. Similar characteristics are shared by many other business problems involving discrete decision-making under uncertainty, making the CCKP formulation widely applicable.

However, despite their practical relevance, chance-constrained optimization problems pose significant computational challenges. They often contain feasible regions that are non-convex, high-dimensional, and overall difficult to characterize, complicating their analysis and practically finding solutions. Classical approaches typically seek to transform probabilistic constraints into deterministic equivalents or tractable approximations [1]. However, traditional methods, including mixed-integer programming formulations [2] and heuristic approximations such as scenario-based approaches [3] and Monte Carlo simulations [4], often scale poorly with problem size. Notably, accurately modeling uncertainty distributions, evaluating constraint violations, and exploring the resulting vast solution spaces leads to prohibitive runtimes or suboptimal solutions.

In parallel, the application of quantum algorithms to knapsack and other combinatorial optimization problems has recently attracted significant attention [5, 6, 7, 8, 9]. Notable examples include Copula-QAOA, which employs warm-started mixers that are biased toward greedy solutions [10], and the Quantum Tree Generator, which facilitates amplitude amplification across all feasible solutions [11]. In addition, recent utility-scale experiments have showcased constrained optimization on systems with up to 150 qubits using shallow mixer circuits [12]. However, to the best of our knowledge, all of these existing gate-based approaches exclusively address problems with deterministic constraints. On the stochastic side, quantum annealing has been applied to chance-constrained problems through scenario-based MILP reformulations [13], demonstrating the viability of quantum approaches for optimization under uncertainty while still relying on classical scenario sampling.

This work introduces a novel quantum-classical framework that extends QAOA-based approaches to CCKPs, leveraging gate-based quantum hardware to tackle stochastic optimization problems. The proposed quantum method employs a variational circuit that embeds chance constraints through a deterministic reformulation, enabling efficient generation of a pool of high-quality

bitstrings that respect a prescribed risk level and exhibit an increased likelihood of maximizing knapsack value. At its core, the circuit architecture takes inspiration from the QAOA-based Lagrangian Duality Discretized Adiabatic (LD-DA) circuit originally proposed for deterministic knapsack problems [14], but it is augmented with a variational component for this stochastic setting.

Additionally, to mitigate known challenges in variational training, such as barren plateaus [15, 16], a parameter transfer strategy based on constraint-alignment is introduced. This enables parameters to be transferred from relatively small problem instances to larger ones, where the exponential growth of the solution space renders direct optimization challenging. These parameters are subsequently refined with a limited number of fine-tuning optimization iterations to obtain samples characterized by sufficiently low objective values, which are then post-processed by a novel self-consistent classical recovery scheme customized for CCKP problems.

Furthermore, large knapsack instances pose substantial challenges for classical solvers. Accordingly, this work includes classical benchmarking on increasingly large CCKPs to assess the onset of computational intractability, motivate the proposed quantum-classical approach, and contextualize performance comparisons. Classical benchmarking experiments obtained using Gurobi were used to compare against quantum hardware experiments conducted on IBM Heron processors, involving problem sizes ranging from 20 to 150 items. This corresponds to circuit depths up to 177 comprising a total of 3443 gates. Overall, the quantum solutions are comparable in quality to classical heuristic methods for all tested instances under the same evaluation criteria.

The remainder of this paper is organized as follows: Sect. 2 introduces the chance-constrained knapsack formulation, the variational quantum circuit construction, the hybrid training, and the post-processing methodology. Sect. 3 presents numerical results from both quantum simulations and hardware experiments alongside classical benchmarks followed by conclusions and outlook in Sect. 4. The appendices (App. A–App. D) provide supporting theoretical proofs, extended analytical derivations, details on mixer behavior and circuit geometry, pseudocode for post-processing and recovery algorithms, and additional experimental and implementation details that supplement the main text.

2 Methods

2.1 Chance-constrained knapsack problem

The classical (deterministic) multi-dimensional knapsack problem extends the standard 0–1 knapsack by imposing multiple constraints. The chance-constrained knapsack problem (CCKP) generalizes this formulation by replacing the deterministic item weights with random variables, which accounts for uncertainty in resource consumption. Specifically, for item $i \in \{1, \dots, n\}$ and constraint $m \in \{1, \dots, M\}$, the random weight $\tilde{w}_i^{(m)}$ is drawn from a distribution family $\Delta(\mu, \sigma)$ with mean $\mu_i^{(m)}$ and standard deviation $\sigma_i^{(m)}$.

The CCKP seeks a binary decision vector $x \in \{0, 1\}^n$, where $x_i = 1$ indicates item i is selected. An optimal x maximizes total value while ensuring constraints on weight capacity are satisfied with high probability. Formally, the problem is defined as:

$$\begin{aligned} \max_{x \in \{0,1\}^n} \quad & \sum_{i=1}^n v_i x_i \\ \text{s.t.} \quad & \mathbb{P}\left(\sum_{i=1}^n \tilde{w}_i^{(m)} x_i \leq C^{(m)}\right) \geq 1 - \epsilon^{(m)}, \quad m \in \{1, 2, \dots, M\}. \end{aligned} \quad (1)$$

For certain weight distributions Δ , the CCKP can be reformulated with deterministic constraints as follows:

Theorem 1. *For the mutually independent knapsack weights $\tilde{w}_i^{(m)}$ chosen from a distributions $\Delta(\mu, \sigma)$ closed under convolution with mean $\mu_i^{(m)}$ and standard deviation $\sigma_i^{(m)}$, and its cumulative distribution function Φ strictly increasing and invertible, the chance-constrained problem in (1) can be rewritten as*

$$\max_{x \in \{0,1\}^n} \quad \sum_{i=1}^n v_i x_i \quad (2)$$

$$\begin{aligned} \text{s.t.} \quad & \sum_{i=1}^n \mu_i^{(m)} x_i + \Phi^{-1}(1 - \epsilon^{(m)}) \sqrt{\sum_{i=1}^n \sigma_i^{(m)2} x_i} \leq C^{(m)} \\ & \forall \quad m \in \{1, 2, \dots, M\} \end{aligned} \quad (3)$$

where Φ^{-1} is the inverse of the cumulative distribution function (CDF) of the distribution $\Delta(0, 1)$ and some small $\epsilon^{(m)} > 0$.²

App. A and [1] provide the proof of this result. As stated in Theorem 1, the distribution Δ must be closed under convolution, with a CDF that is strictly increasing and invertible. Several commonly used distributions belonging to the family of α -stable distributions including Normal, Cauchy, and, Lévy satisfy these properties. In this paper, we restrict our attention to item weights drawn from Normal distributions.

2.2 Classical problem instance creation

Classical benchmarks are constructed using a deterministic mixed-integer second-order cone program (MISOCP) reformulation of the CCKP in (3) by squaring

²For distributions satisfying the stated conditions, the probabilistic constraint in (1) admits an exact reformulation as the second-order cone constraint in (3). Classical solvers such as Gurobi handle these constraints by squaring both sides to obtain an equivalent quadratic constraint, enabling the use of second-order cone programming (SOCP) [1].

both sides. This representation isolates the interaction between combinatorial decision variables and quadratic second-order cone constraints, enabling systematic exploration of regimes that exhibit significant classical computational difficulty. Hard problem instances are generated and solved using Gurobi 12.0.2 [17] on an Intel Xeon Silver 4310 CPU (2.10 GHz) with 12 cores. Problem instances are chosen to lie within regimes accessible to both classical and quantum hardware, specifically from combinations of $n \in \{20, 40, 75, 100, 150\}$ and $M \in \{1, 5, 20, 50, 100\}$ in conjunction with the following additional parameters:

1. Maximum item value $v_{\max} \in \{4, 20, 50, 100\}$ as the upper bound for uniform sampling of item values $v_i \in (0, v_{\max}]$.
2. Maximum item weight mean $\mu_{\max} \in \{5, 25, 50, 100\}$, where individual item weight means $\mu_i^{(m)}$ are sampled uniformly from the interval $(0, \mu_{\max}]$.
3. Item weight standard deviation σ , where for all items, the variance $\sigma^2 \in \{0.5, 1, 1.5, 2, 3\}$.
4. Approximate constraint density ratio $r_D \in \{0.25, 0.5, 0.75\}$, where $r_D \approx \frac{\sum_{i=1}^n \mathbf{1}(\mu_i^{(m)} \neq 0)}{n}$, and this ratio informs a random subset selection of $\mu_i^{(m)}$ to be set to zero, together with the corresponding subset of $\sigma_i^{(m)}$.
5. Approximate capacity ratio $r_C \in \{0.25, 0.5, 0.75\}$, where $r_C \approx \frac{\sum_{i=1}^n \mu_i^{(m)}}{C^{(m)}}$, in order to determine $C^{(m)}$ after the parameters $\mu_i^{(m)}$ have been specified according to the rules outlined above.
6. Fixed value for $\epsilon^{(m)} = 0.2$ for all m and henceforth denote $\epsilon^{(m)} \equiv \epsilon$. This permits comparisons in algorithm performance across instances rather than changes in risk tolerance.

In the first optimization phase, a comprehensive parameter sweep is performed over multiple dimensions characterizing the stochastic knapsack problem, resulting in a total of 3,900 instances. Each instance is allotted a maximum runtime of three minutes, and instances which Gurobi could prove optimality within this time limit are labeled as “easy.” Empirical results (see Fig. 2a) indicate that problem difficulty is most strongly affected by increasing values of n followed by M , with additional sensitivity to smaller values of $\mu_{\max}$. Larger values of σ^2 further increase runtime and exacerbate optimality gaps, whereas $v_{\max}$, r_C , and r_D have negligible impact. Empirically, the parameter configuration $v_{\max} = 50$, $\mu_{\max} = 5$, $r_C = 0.25$, and $r_D = 0.5$ consistently yield challenging instances as n and M increased. Accordingly, these parameter values are fixed for the following two additional phases.

The second optimization phase conducts a limited exploration of the weight variance σ^2 . In the third phase, σ^2 is fixed to $\sigma^2 = 1$ while investigating solver performance as the number of items scales. Sixteen instances are created by varying $n \in \{20, 40, 50, 75, 100, 150, 500, 1000\}$ and $M \in \{5, 20\}$. The problem

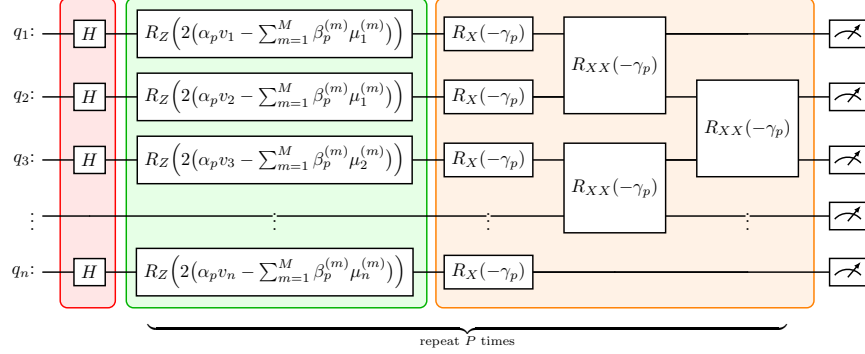


Figure 1: QAOA-based variational quantum circuit with P layers and three variational parameters $\theta = (\alpha, \beta, \gamma) \in \mathbb{R}^{P \times (2+M)}$. The circuit contains parametrized single qubit X and two-qubit XX rotations where two-qubit gates are arranged over a line. The phase separation block applies diagonal $R_Z(2(\alpha_p v_i - \sum_{m=1}^M \beta_p^{(m)} \mu_i^{(m)}))$ gates, where layers are indexed by $p = 1, \dots, P$.

instances generated in this step are used to assess the performance of multiple classical solvers, as summarized in Fig. 2. From this set, only instances with $n \leq 500$ are retained as candidates for quantum simulation and hardware execution, forming the set of problems addressed by the quantum-classical algorithm experiments below.

2.3 Quantum circuit construction

To evaluate these instances on quantum hardware, a circuit ansatz is required that mirrors the structure of the underlying optimization problem while remaining compatible with gate-based devices. Accordingly, the proposed quantum-classical algorithm employs a QAOA-based [18] variational ansatz with Z , X and XX rotations, as shown in Fig. 1. This construction is inspired by the Trotterized time-evolution operator arising from the Lagrangian Dual-Discretized Adiabatic Quantum Computation (LD-DAQC) scheme that has exhibited improved performance relative to canonical QUBO-based circuits on a large set of knapsack problems [14]. The ansatz incorporates knapsack item values and mean weights into Z -gate rotations with the mixer as single X and two-qubit XX gates.

Formally, the linear geometry ansatz can be split into two Hamiltonians, H_{problem} and H_{mixer} , defined as

$$H_{\text{problem}}^{(p)} = \sum_{i=1}^n \left(\alpha_p v_i - \sum_{m=1}^M \beta_p^{(m)} \mu_i^{(m)} \right) Z_i, \quad (4)$$

$$H_{\text{mixer}}^{(p)} = -\frac{1}{2} \sum_{i=1}^n (\gamma_p) X_i - \frac{1}{2} \sum_{i=1}^{n-1} (\gamma_p) X_i X_{i+1}, \quad p \in \{1, \dots, P\} \quad (5)$$

for variational parameters α, β, γ and each item i , constraint m , and QAOA layer p . The linear two-qubit entanglement map requires relatively low qubit connectivity and makes it well-suited for IBM Quantum processors with heavy-hex and square lattice qubit topology.

2.3.1 Toy example

While the full circuit structure is illustrated in Fig. 1, its effect on constrained knapsack dynamics is most easily understood in a minimal setting. To build intuition, the role of Z -diagonal phase separator $H_{\text{problem}}^{(p)}$ and the $X+XX$ mixer terms in $H_{\text{mixer}}^{(p)}$ are first analyzed through a minimal two-state toy example using a minimal two-state toy example with a single constraint and linear geometry ($M = 1$).³

Two-state subspace. Consider a knapsack of n items arranged on a cycle with values v_j and weights μ_j for $j = 1, \dots, n$. Focusing on two adjacent items j and $j+1$ on the mixer graph, we examine two computational-basis states $|a\rangle = |1_j 0_{j+1}\rangle$ and $|b\rangle = |0_j 1_{j+1}\rangle$ that differ only on these qubits while all other selections remain identical. The states $|a\rangle$ and $|b\rangle$ are connected by an XX swap ($|10\rangle \leftrightarrow |01\rangle$) along the edge $(j, j+1)$. Restricting to this two-state subspace, the uniform superposition induced by the initial state $|+\rangle^{\otimes n}$ takes the form

$$|\psi_0\rangle = \frac{1}{\sqrt{2}}(|a\rangle + |b\rangle). \quad (6)$$

Acting on $|\psi_0\rangle$, the Z -diagonal phase separator imparts opposing phases on the two configurations, yielding

$$|\psi_1\rangle = \frac{1}{\sqrt{2}}(e^{-i\delta/2}|a\rangle + e^{+i\delta/2}|b\rangle), \quad (7)$$

where the *phase gap*,

$$\delta \equiv E_a - E_b = 2[\alpha(v_{j+1} - v_j) - \beta(\mu_{j+1} - \mu_j)], \quad (8)$$

is determined by the difference in value and weight contributions between the two states.

When restricted to the $\{|a\rangle, |b\rangle\}$ subspace, the XX interaction acting on edge $(j, j+1)$ reduces to a $-\sigma_x$ operator (negative sign due to Eq. (5)), as it swaps $|10\rangle \leftrightarrow |01\rangle$. Therefore, the mixer unitary on this subspace becomes

$$e^{i\gamma\sigma_x} = \cos \gamma I + i \sin \gamma \sigma_x. \quad (9)$$

³We omit in this section the superscripts (m) .

Applied to $|\psi_1\rangle$, the mixer produces the final state

$$|\psi_2\rangle = \frac{1}{\sqrt{2}} \begin{pmatrix} \cos \gamma e^{-i\delta/2} + i \sin \gamma e^{+i\delta/2} \\ \cos \gamma e^{+i\delta/2} + i \sin \gamma e^{-i\delta/2} \end{pmatrix}. \quad (10)$$

As a result, the measurement probabilities take the form

$$\begin{aligned} \mathbb{P}(a) &= \frac{1}{2} |\cos \gamma e^{-i\delta/2} + i \sin \gamma e^{+i\delta/2}|^2 = \frac{1}{2} (1 - \sin 2\gamma \sin \delta), \\ \mathbb{P}(b) &= \frac{1}{2} (1 + \sin 2\gamma \sin \delta), \end{aligned} \quad (11)$$

where the population imbalance between the two configurations is given by

$$\mathbb{P}(b) - \mathbb{P}(a) = \sin 2\gamma \sin \delta, \quad (12)$$

When returning to the full problem, feasibility constraints reemerge: the capacity C enters implicitly through a penalty term in the classical loss function rather than through the quantum circuit itself. As a result, feasibility requirements constrain how the classical optimizer selects the variational parameters α , β , and γ , linking the capacity to probability transfer. To make this relationship explicit, we now turn to the expected weight distribution, written as

$$\langle W \rangle = \mathbb{P}(a) W_a + \mathbb{P}(b) W_b, \quad (13)$$

where $W_a = W_{\text{rest}} + \mu_j$ and $W_b = W_{\text{rest}} + \mu_{j+1}$ are the total weights of the two item selections and $W_{\text{rest}} = \sum_{\ell \in S \setminus \{j\}} \mu_\ell$ is the weight of the items common to both states. Substituting measurement probabilities from Eq. 11, we get:

$$\langle W \rangle = W_{\text{rest}} + \frac{\mu_j + \mu_{j+1}}{2} + \frac{\mu_{j+1} - \mu_j}{2} \sin 2\gamma \sin \delta. \quad (14)$$

Additionally, the *marginal residual*, defined as

$$\hat{r} = \left(W_{\text{rest}} + \frac{\mu_j + \mu_{j+1}}{2} \right) - C, \quad (15)$$

measures the constraint violation after the fixed items and the average weight of the two swappable items. The feasibility condition $\langle W \rangle \leq C$, equivalent to $\hat{r} \leq 0$, therefore enforces a direct constraint on the allowable probability transfer between the two configurations:

$$\sin 2\gamma \sin \delta \leq \frac{2\hat{r}}{\mu_j - \mu_{j+1}} \quad (\text{when } \mu_j < \mu_{j+1}). \quad (16)$$

When $\mu_j < \mu_{j+1}$, the right-hand side of (16) is decreasing in $\hat{r}$, implying that larger marginal residuals increasingly restrict feasible probability transfer, eventually forcing a reversal of flow.

Under the same distribution, the expected value is

$$\langle V \rangle = \frac{V_a + V_b}{2} + \frac{v_{j+1} - v_j}{2} \sin 2\gamma \sin \delta, \quad (17)$$

which monotonically increases in $\sin 2\gamma \sin \delta$ when $v_{j+1} > v_j$. The optimizer can maximize value by pushing the amplitude transfer to the capacity limit and its optimal operating point saturates the bound

$$\sin 2\gamma^* \sin \delta^* = \min\left(1, \frac{2\hat{r}}{\mu_j - \mu_{j+1}}\right). \quad (18)$$

When $\hat{r} \leq (\mu_j - \mu_{j+1})/2$ the minimum saturates at 1 and the optimizer achieves full transfer unconstrained by capacity. When $(\mu_j - \mu_{j+1})/2 < \hat{r} \leq 0$, the bound is active and capacity throttles the transfer – the optimizer must reduce γ or increase β/α to stay feasible. When $\hat{r} > 0$, the average weight already exceeds capacity, feasibility requires $\sin 2\gamma \sin \delta < 0$, and the optimizer reverses the amplitude flow toward the lighter item even at the cost of value.

The three $\hat{r}$ -based regimes of the toy model and the optimal operating point (18) carry over to the chance-constrained variant with one modification: $\hat{r}$ is replaced by the *constraint residual*

$$r(\mathbf{x}) = L(\mathbf{x}) - C, \quad (19)$$

where $L(\mathbf{x})$ is the left-hand side of (3). Because the variance term $\sqrt{\sum \sigma_j^2 x_j}$ in $L(\mathbf{x})$ is not separable across qubits, r cannot be encoded in per-item Z -rotations. This gap is bridged by the loss function (23), which evaluates r on each sample and feeds the violation signal back to the optimizer. Over successive iterations, this drives β^*/α^* above the deterministic Lagrange multiplier, compensating for the variance risk that the circuit cannot represent directly, until the probability transfer on each boundary edge matches the operating point (18) with $\hat{r} \rightarrow r$.

2.3.2 The role of H_{mixer}

The effectiveness of the ansatz is ultimately dictated by the behavior of the H_{mixer} with $X+XX$ components in Eq. (5). Its role can be evaluated by continuing to examine the simplified CCKP instance with a single constraint ($M = 1$) and homogeneous weight parameters $\mu_i = \mu$, $\sigma_i = \sigma$ for all items. Substituting into the deterministic reformulation of Theorem 1, the chance constraint becomes

$$\mu k + z_\epsilon \sigma \sqrt{k} \leq C, \quad (20)$$

where $k = \sum_{i=1}^n x_i$ is the number of selected items and $z_\epsilon = \Phi^{-1}(1 - \epsilon)$. Substituting $u = \sqrt{k}$ yields the quadratic $\mu u^2 + z_\epsilon \sigma u - C \leq 0$, whose positive root gives the maximum feasible cardinality

$$k^* = \left\lfloor \left(\frac{-z_\epsilon \sigma + \sqrt{z_\epsilon^2 \sigma^2 + 4\mu C}}{2\mu} \right)^2 \right\rfloor. \quad (21)$$

The optimization problem therefore reduces to selecting the k^* items with the largest values v_i . The constraint determines *how many* items to select; the

choice of *which* items is a combinatorial optimization within the set of k^* -element subsets. Note, in general, except in this simplified case, the optimal cardinality k^* is not known beforehand.

The initial state $|+\rangle^{\otimes n}$ (see Fig. 1) distributes amplitude across Hamming-weight sectors according to the binomial distribution $\mathbb{P}(k) = \binom{n}{k}/2^n$, which is sharply peaked around $k = n/2$ with the bitstring subspace $\mathcal{H}_k = \text{span}\{|x\rangle : |x| = k\}$. When the feasible cardinality k^* is far from this peak, as is typical under restrictive capacity constraints, the amplitude initially assigned to $\mathcal{H}_{k^*}$ is exponentially suppressed. Consequently, an effective mixer must facilitate the redistribution of amplitude between different Hamming-weight sectors in order to realize feasible solutions.

The necessary components for this redistribution are contained within the single-qubit and two-qubit interactions of H_{mixer} . The single-qubit X term allows transitions between $\mathcal{H}_k$ and $\mathcal{H}_{k\pm 1}$, while the XX term enables swaps within a fixed Hamming-weight sector ($|01\rangle \leftrightarrow |10\rangle$, $\Delta k = 0$) and between sectors differing by two ($|00\rangle \leftrightarrow |11\rangle$, $\Delta k = \pm 2$). Together, these components promote amplitude propagation between Hamming-weight sectors ($\mathcal{H}_k$ to $\mathcal{H}_{k\pm 2}$), enabling access to the entire solution space.

In App. B, we extend our toy-example analysis in Sect. 2.3.1 to XX interactions on a linear geometry. We deduce that for the state $|\mathcal{T}\rangle$, which denotes the computational-basis state corresponding to the k^* -element subset $\mathcal{T} \subseteq \{1, \dots, n\}$, the measurement probability becomes

$$\mathbb{P}(\mathcal{T}) \approx |\omega_{\mathcal{T}}|^2 \left(1 + 2\gamma \sum_{\substack{(j,j') \in E(G): \\ j \in \mathcal{T}, j' \notin \mathcal{T}}} \sin(2(\phi_j - \phi_{j'})) \right), \quad (22)$$

where $\phi_j = \alpha v_j - \beta \mu_j$ denotes the effective rotation, $j \in \mathcal{T}$ is the included item, and $j' \notin \mathcal{T}$ is its excluded neighbor on the linear mixer graph.

In essence, equation (22) shows that the XX chain acts as a bank of parallel reduced-cost comparators. Each edge (j, j') in the mixer graph contributes a term $\sin(2(\phi_j - \phi_{j'}))$ that tests whether keeping item j versus swapping it for item j' is favorable, given the current value-weight trade-off set by α and β . If the included item has a more favorable value-weight balance ($\alpha v_j - \beta \mu_j > \alpha v_{j'} - \beta \mu_{j'}$), the contribution is positive and the probability of $\mathcal{T}$ increases. Conversely, when the excluded item is favored, amplitude flows away from $\mathcal{T}$ toward the swapped configuration. The total probability boost for a subset $\mathcal{T}$ is the sum over all boundary edges, defined as edges with one endpoint in $\mathcal{T}$ and one outside. States whose included items consistently outperform their excluded neighbors accumulate positive contributions from multiple edges simultaneously, concentrating amplitude on high-quality solutions.

When a capacity constraint $\sum_{j \in \mathcal{T}} \mu_j \leq C$ is present, the amplitude transfers on different boundary edges are no longer independent. The marginal residual $\hat{r}(\mathcal{T}) = \sum_{j \in \mathcal{T}} \mu_j - C$ limits how much weight can be redistributed across the boundary of $\mathcal{T}$: a swap that replaces item j with a heavier item j' is only feasible

if $\mu_j - \mu_{j'} \geq \hat{r}(\mathcal{T})$. At the circuit level, the phase separator encodes this through the weight penalty β , where edges whose swap would violate the capacity bound receive a large negative phase shift, automatically suppressing their amplitude contribution in Eq. (22). Thus, the same mechanism identified in the two-state case, where the interplay between the phase gap δ and the marginal residual $\hat{r}$, extends to the full chain. However, here the boundary edges collectively share a single capacity budget, coupling their amplitude transfers through the aggregate feasibility condition $\sum_{j \in \mathcal{T}} \mu_j \leq C$.

2.3.3 Circuit Parameters

Loss function. Although the structure of the ansatz naturally encodes feasibility through its phase and mixing dynamics, its performance ultimately depends on how the variational parameters are selected and optimized. Practically, key considerations include training stability and expressivity, and stability in particular is known to introduce barren plateau issues in variational schemes [15]. Therefore, variational training is restricted to relatively small circuits with a loss function defined as

$$f(\mathbf{x}_\theta) = -\mathbf{v}^\top \mathbf{x}_\theta + \lambda \max(0, \sum_{m=1}^M L_m(\mathbf{x}_\theta; \boldsymbol{\mu}_m, \boldsymbol{\sigma}_m) - C^{(m)}), \quad (23)$$

where

$$L_m(\mathbf{x}_\theta; \boldsymbol{\mu}_m, \boldsymbol{\sigma}_m) := \sum_{i=1}^n \mu_i^{(m)} x_i + \Phi^{-1}(1 - \epsilon^{(m)}) \sqrt{\sum_{i=1}^n \sigma_i^{(m)2} x_i} \quad (24)$$

refers to the left hand side in (3).

Instead of minimizing the empirical mean, the optimizer employs conditional value-at-risk (CVaR(ξ)) [19] computed over samples collected from the quantum processor, with parameters updated using COBYLA [20, 21]. In general, CVaR may be defined with respect to either the lower or upper ξ -tail of the cost distribution, corresponding to an emphasis on the most favorable or most adverse outcomes, respectively. CVaR(ξ)-based optimization is empirically observed to demonstrate improved convergence characteristics and enhanced robustness to hardware noise [19, 22]. In this work, we adopt the lower-tail CVaR.

A close examination of the loss in (23) shows that the penalty term always contributes positively to the loss, allowing the multiplier λ to be chosen as a sufficiently large positive value. However, the constraints may still be violated because the first term, $-\mathbf{v}^\top \mathbf{x}_\theta$, is negative. Setting $\lambda = \sum_i v_i$ guards against the largest possible reduction in the loss that can be caused by the term $-\mathbf{v}^\top \mathbf{x}_\theta$.

Parameter transfer. For larger problem instances, optimized parameters are either directly transferred from smaller instances or employed to warm start a limited number of variational optimization steps on the larger circuits. The employed parameter transfer scheme takes into account characteristics of the

constraints to align the transfer of parameters from circuits with fewer qubits ($\mathcal{C}_{\text{src}}$) to circuits with a larger number of qubits ($\mathcal{C}_{\text{tgt}}$).

Each constraint m is characterized by a feature vector

$$\mathbf{f}^{(m)} = [\bar{\mu}^{(m)}, \bar{\sigma}^{(m)}, r_C^{(m)}]^\top \in \mathbb{R}^3,$$

constructed from per-constraint normalized weights. Specifically, the parameters $\mu_i^{(m)}$ and $\sigma_i^{(m)}$ are rescaled by the root-mean-square magnitude of $\{\mu_i^{(m)}\}_{i=1}^n$ within each constraint m , yielding scale-invariant statistics. Here, $\bar{\mu}^{(m)}$ and $\bar{\sigma}^{(m)}$ denote the mean normalized weight and standard deviation, and

$$r_C^{(m)} = \frac{\sum_{i=1}^n \mu_i^{(m)}}{C^{(m)}}$$

is the capacity ratio, which remains invariant under this normalization.

Given feature matrices $\mathbf{F}_{\text{src}}, \mathbf{F}_{\text{tgt}} \in \mathbb{R}^{M \times 3}$, we solve the linear assignment problem $\pi^* = \arg \min_{\pi} \sum_{i=1}^M \|\mathbf{f}_{\text{tgt}}^{(i)} - \mathbf{f}_{\text{src}}^{(\pi(i))}\|_2$ via the Hungarian algorithm [23] in $\mathcal{O}(M^3)$ time, then permute the source circuit parameter β matrix by π^* before transfer (see Fig. 1). This ensures each target constraint inherits parameters from the most structurally similar source constraint.

2.4 Simulation and hardware runs

With the circuit architecture and parameter transfer strategy established, the next step is to implement the resulting ansatz via matrix product state (MPS) simulations and gate-based quantum hardware sampling.

Simulations were carried out using the Qiskit **SamplerV2** [24] interface with the MPS backend [25, 26], which efficiently approximates circuit execution by truncating entanglement according to a fixed bond dimension. A maximum bond dimension of 512 and a truncation threshold of 10^{-6} were used to balance simulation accuracy and computational cost. All simulations matched the qubit count of the corresponding knapsack instances and employed fixed random seeds for reproducibility.

Hardware runs were conducted on IBM Heron processors, including **ibm_fez**, **ibm_pittsburgh**, and **ibm_aachen**, with circuits characterized by 20-150 qubits that feature up to 3443 gates with total circuit depth up to 177. The two-qubit gates of the ansatz in Fig. 1 are arranged along a linear chain, and these entangling operations are then mapped to the native heavy-hex connectivity of the IBM quantum device. Additionally, the circuit is transpiled using the Qiskit Primitive V2 transpiler [24] with the optimization level set to 3. Dynamical decoupling is employed for error suppression using the default ‘XX’ sequence [27]. The resulting transpiled circuit is then executed on the quantum processor using the Qiskit sampling primitive, with $N_{\text{shots}} = 2^{15}$ measurement samples collected.

2.5 Self-consistent post-processing

Samples produced by quantum hardware are inherently noisy, which can significantly degrade solution quality for constrained combinatorial optimization problems. To mitigate these effects, a novel problem-aware classical post-processing procedure is applied to the raw measurement outcomes. As described below and in pseudocode in App. C, this method iteratively refines noisy samples by exploiting problem structure and preferentially applying trial bit flips that lead to improved objective values while maintaining feasibility.

This method draws inspiration from the configuration recovery scheme used for error mitigation in quantum chemistry calculations [28]. The procedure alternates between a residual-guided sample repair step and updates to a *selection-probability vector* $\mathbf{p} \in [0, 1]^n$, where each component p_i estimates the frequency with which item i appears in high-quality feasible solutions. This probability vector is maintained by an outer iteration loop and fed back into the repair heuristic, closing a self-consistent feedback loop that progressively improves the recovery set. To make this procedure precise, the recovery problem can be formalized as follows.

Recovery problem. Given the knapsack objective $f(\mathbf{x}) = \mathbf{v}^\top \mathbf{x}$, following (19) the constraint residual of constraint m is

$$r^{(m)}(\mathbf{x}) = \sum_{i=1}^n \mu_i^{(m)} x_i + \Phi^{-1}(1 - \epsilon^{(m)}) \sqrt{\sum_i (\sigma_i^{(m)})^2 x_i} - C^{(m)} \quad (25)$$

and a sample is feasible if and only if $r^{(m)}(\mathbf{x}) \leq 0$ for all m . Sample quality is measured by the penalty-augmented SOCP objective

$$\mathcal{L}(\mathbf{x}) = -f(\mathbf{x}) + \lambda \sum_m g^{(m)}(\mathbf{x}) \quad (26)$$

where $g^{(m)}(\mathbf{x}) = \max(0, r^{(m)}(\mathbf{x}))$ and the repair heuristic can be viewed as greedily decreasing $\mathcal{L}$ under single-bit moves.

Two-phase sample repair. At the start of each iteration, the sample pool is partitioned into a feasible subset $\mathcal{S}_{\text{feas}}$ and an infeasible subset $\mathcal{S}_{\text{infeas}}$ according to the residuals (25). Every sample in $\mathcal{S}_{\text{feas}} \cup \mathcal{S}_{\text{infeas}}$ is passed through a two-phase repair procedure that operates through single-bit flips.

Phase 1 (feasibility restoration) For samples drawn from $\mathcal{S}_{\text{infeas}}$, the first phase removes items by flipping selected bits from 1 to 0 while any constraint is violated. For each currently selected item i , let $\mathbf{x}^{(-i)}$ denote the vector obtained from $\mathbf{x}$ by setting x_i to zero, and define the relief as

$$\text{relief}_i(\mathbf{x}) = \sum_m g^{(m)}(\mathbf{x}) - \sum_m g^{(m)}(\mathbf{x}^{(-i)}). \quad (27)$$

Among items with positive relief, Phase 1 flips the bit of the one maximizing

$$\text{score}_i = \frac{\text{relief}_i(\mathbf{x})}{\max(v_i, \delta)} + \eta(1 - p_i), \quad (28)$$

where $\delta > 0$ is a small constant that prevents division by zero for zero-value items and $\eta \geq 0$ sets the strength of the probability bias. The first term rewards relief per unit of lost profit; the second nudges the heuristic toward flipping items that are rarely selected in the current probability model (i.e., those with low p_i). If no bit flip yields positive relief, the phase terminates early, leaving the sample unchanged.

Phase 2 (greedy improvement) Phase 2 applies to every sample that is feasible at the end of Phase 1: samples that skipped Phase 1 drawn from $\mathcal{S}_{\text{feas}}$ and those repaired from $\mathcal{S}_{\text{infeas}}$. For each sample, Phase 2 flips unselected bits from 0 to 1 in decreasing order of v_i , accepting each flip only if the resulting sample remains feasible. Since the square-root term in (25) is non-separable, to determine feasibility, all M constraints are checked. In other words, the marginal contribution of flipping item i from 0 to 1 in constraint m is

$$\delta r_i^{(m)}(S) = \mu_i^{(m)} + \Phi^{-1}(1 - \epsilon^{(m)})[\sqrt{S^{(m)} + (\sigma_i^{(m)})^2} - \sqrt{S^{(m)}}], \quad (29)$$

with $S^{(m)} = \sum_j (\sigma_j^{(m)})^2 x_j$.

Objective-weighted probability update. After repair, feasible samples receive softmax weights $w_k \propto \exp(f(\mathbf{x}_k) - f_{\max})$, where subtracting the maximum profit $f_{\max}$ ensures numerical stability. The selection probabilities are then updated by an average over K random batches $\mathcal{B}_1, \dots, \mathcal{B}_K$ of the samples:

$$p_i^{\text{new}} = \frac{1}{K} \sum_{\ell=1}^K \frac{\sum_{\mathbf{x} \in \mathcal{B}_\ell} w_{\mathbf{x}} x_i}{\sum_{\mathbf{x} \in \mathcal{B}_\ell} w_{\mathbf{x}}}, \quad i = 1, \dots, n. \quad (30)$$

Batching reduces the variance of $\mathbf{p}^{\text{new}}$ relative to a single global weighted mean and makes the estimate more robust when the feasible set is small. If no repaired sample is feasible, as may occur during initial iterations for highly noisy pools, the full repaired set is used as a fallback to ensure that (30) remains well-defined.

Self-consistent loop. The outer loop repeatedly (i) repairs every sample in the current pool, (ii) recomputes weights and probabilities on the feasible subset, and (iii) re-partitions the repaired pool into feasible and infeasible subsets for the next iteration. No fresh samples are drawn from $\mathbf{p}^{\text{new}}$; the scheme is a deterministic refinement of the original pool of samples, with the probabilities entering the next repair step only through the score (28). Iteration stops when $\Delta = \frac{1}{n} \|\mathbf{p}^{\text{new}} - \mathbf{p}\|_1 < \epsilon_{\text{conv}}$ or after $\mathcal{T}$ iterations, with per-iteration cost $O(|\mathcal{S}| \cdot (F_{\text{rem}} + F_{\text{add}}) \cdot n \cdot M)$.

2.6 Computational cost.

The quantum-classical scheme involves three steps.

The **first** step, using COBYLA, optimizes circuit parameters θ with an MPS simulator on a relatively small, surrogate problem with N_{eval}^S function evaluations yielding θ^S after convergence.

The **second** step transfers θ^S to the larger circuit corresponding to the target problem to be solved and subsequently tunes, again with COBYLA, the parameters with N_{eval} ($N_{\text{eval}} \ll N_{\text{eval}}^S$) function evaluations either with an approximate MPS simulator or on quantum hardware yielding θ^* . This is followed by a final sampling run with the circuit initialized to θ^* . As a result, the large, target circuit’s execution time is $\propto N_{\text{shots}} N_{\text{eval}}$ while the parameter training on the surrogate problem is a one-time $\propto N_{\text{shots}}^S N_{\text{eval}}^S$, as the same parameters can be transferred to circuits corresponding to multiple target problems sharing similar features.

Finally, the **third** step consists of classical recovery run for $\mathcal{T}$ iterations using $|\mathcal{S}|$ unique bitstrings⁴ to refine noisy hardware samples obtained from the final sampling run on the circuit in the second step. As detailed in the prior section, the recovery step has a runtime $\propto \mathcal{T} \cdot |\mathcal{S}|$. Each recovery iteration performs $|\mathcal{S}|$ repair calls, each of which evaluates at most $\mathcal{O}((F_{\text{rem}} + F_{\text{add}}) \cdot M \cdot n)$ constraint checks with a corresponding probability update at the cost of $\mathcal{O}(|\mathcal{S}| \cdot n)$.

Overall, as hardware sample quality improves through additional parameter training and reduced noise, the recovery step is expected to become less resource-intensive. Higher-fidelity sample distributions reduce recovery costs in terms of both sample complexity and iteration count. The number of function evaluations N_{eval} , which limits the extent of circuit refinement during recovery, therefore serves as a key control parameter for the total runtime.

3 Results

Classical benchmarking. Table 1 presents average Gurobi runtime trends observed over different problem sizes n , number of constraints M , and maximum item weight mean μ_{max} for the 3,900 instances discussed in section Sect. 2.2. Overall, the results show a pronounced runtime inflection point at approximately 75 items to reach optimality, and problems with more constraints exhibit longer runtimes compared to those with fewer constraints.

Fig. 2a presents Gurobi optimality gap evolution over time, capped to a maximum runtime of 30 minutes. These results are reported for “hard” instances obtained by following a systematic scan over different parameters as described in Sect. 2.2. The figure demonstrates that relatively large problems (i.e., $n \in \{500, 1000\}$) maintain higher gaps even after 30 minutes, indicating difficulty continuously increasing with problem size. Conversely, for small knapsack instances, Gurobi consistently proves optimality within 30 minutes. For

⁴There may be repetitions among the N_{shots} samples.

n	$\mathbb{P}(t_o < 180)$	$\bar{t}_r$	M	$\mathbb{P}(t_o < 180)$	$\bar{t}_r$
20	100%	0.1	1	98%	7
40	97%	10	5	87%	33
75	69%	73	20	68%	71
100	56%	103	50	55%	95
150	36%	125	100	50%	106

$\mu_{\max}$	$\mathbb{P}(t_o < 180)$	$\bar{t}_r$
5	52%	94
{25, 50, 100}	76%	51

Table 1: Statistics for average classical benchmarking runtimes $\bar{t}_r$ and the time taken to prove optimality t_o using Gurobi, both times measured in seconds. $\mathbb{P}(t_o < 180)$ denotes the proportion of instances that were solved to proven optimality within 180 seconds. The optimality gap is defined as $f_{\text{ub}}/f_{\text{best}} - 1$, where f_{ub} and f_{best} denote the best upper bound and the best known feasible solution, respectively.

more difficult instances, the Gurobi-best solution is typically identified in under 30 minutes; however, extended runtimes and manual intervention—primarily through parameter tuning—are often required to reduce the optimality gap. For the most challenging instances, substantial user intervention is necessary to obtain the Gurobi-best solution, and runtimes up to 30 hours may be required to further tighten the gap. Although runs could be extended beyond this limit, diminishing returns were observed when attempting to fully close the optimality gap.

While Gurobi serves as an exact baseline, classical hardness is further characterized by comparing its performance against that of several classical heuristic solvers optimizing the same loss function (see Eq. (23)). Fig. 2b indicates closeness κ of heuristic solutions to the best solutions produced by Gurobi, where $\kappa = f_{\text{solver}}/f_{\text{best}} \times 100$ is defined as the best-so-far item-value ratio of the best feasible bitstring obtained by the classical heuristics f_{solver} to the best f_{best} obtained using Gurobi. The heuristics recover the proven optimum on the smallest instances ($n \leq 50$, $M = 5$) and the best-performing heuristic remains within 5% of the Gurobi reference on every instance considered. Performance nevertheless degrades with increasing item count and, more markedly, with the number of constraints: at $n \geq 500$ the spread across solvers widens considerably (e.g., 73–97% at $n = 1000$, $M = 5$), with tabu search proving the most robust ($\kappa \geq 93\%$ throughout). Notably, across all problem instances, heuristics never exceed the Gurobi-best objective.

Quantum-classical scheme. Table 2 summarizes the performance of the proposed quantum scheme across knapsack instances of varying size, alongside the circuit resources required for each configuration. Solution quality is reported in terms of closeness κ to Gurobi’s optimum, enabling a direct com-

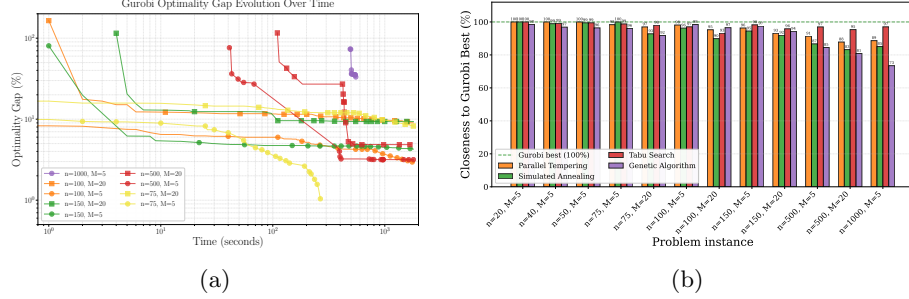


Figure 2: (a) Gurobi optimality gap evolution over time for different problem instances when Gurobi runtime is capped at 30 minutes. The plot shows gap convergence on a log-log scale, where different colors represent problem sizes (n) and marker shapes indicate the number of constraints (M): circles for $M = 5$, squares for $M = 20$. (b) Final solution quality comparison across all solvers after 1800 seconds for different problem instances. The y-axis shows closeness (κ) to Gurobi’s best solution as a percentage, where 100% (green dashed line) indicates matching Gurobi’s performance. Note, to obtain the “Gurobi-best” solutions, the 30 minute time limit was relaxed. Results are sorted by n and M .

Table 2: Performance and circuit resources of the quantum scheme on knapsack instances of size n , with 5 constraints in most cases, except the instance labeled 100(20), which includes 20 constraints. Closeness κ (%) is measured against Gurobi’s best solution; the quantum value is from the best sampled bitstring and the classical value is from the best-performing heuristic among Parallel Tempering, Tabu Search, Genetic Algorithm, and Simulated Annealing. Circuit metrics are the number of variational layers p , total circuit depth, two-qubit gate depth, and total gate count. For $n < 100$, p was chosen by sweeping layer counts and selecting the value that minimized CVaR; for $n \geq 100$, p was fixed by transferring parameters optimized on the 50-item instance.

n	Closeness κ (%)		Circuit resources			
	Quantum	Classical heuristic	p	Total depth	2Q depth	Gate count
20	100.0	100.00	9	55	35	571
40	100.0	100.00	8	71	53	1032
50	100.0	100.00	7	77	61	1143
75	98.6	100.00	10	114	92	2390
100	97.2	98.5	7	127	111	2293
100(20)	96.9	96.5	7	127	111	2293
150	94.9	98.3	7	177	161	3443

parison between the best sampled bitstring from the quantum solver and the strongest classical heuristic identified per instance among Parallel Tempering, Tabu Search, Genetic Algorithm, and Simulated Annealing.

For small instance sizes ($n \leq 50$), both the quantum scheme and the classical heuristic recover Gurobi’s best ($\kappa = 100\%$), indicating that these regimes are easily tractable for both approaches. At larger scales, the classical heuristic generally maintains a slight edge in solution quality. At $n = 75$, the classical method still achieves $\kappa = 100\%$, while the quantum scheme attains 98.6%. A similar trend persists at $n = 100$ with 5 constraints and $n = 150$, where the classical heuristic reaches 98.5% and 98.3%, compared to 97.2% and 94.9% for the quantum scheme, respectively. However, the quantum approach marginally outperforms the classical heuristic in the more tightly constrained $n = 100$ case with 20 constraints (96.9% vs. 96.5%). These results indicate that the quantum scheme remains competitive as problem size grows suggests that the quantum scheme is particularly competitive as constraint density increases.

The circuit depths reported in Table 2 induce a rapidly increasing classical simulation cost when using MPS methods, since the runtime scales polynomially with both the circuit size and the MPS bond dimension [29, 30]. For circuits of fixed depth, the dominant contributions to the runtime is governed by the bond dimension alone. Here, the circuit employed contains a linear, one-dimensional entanglement structure whose bond dimension grows linearly with the number of items in the knapsack instance. As a result, problem instances exceeding approximately 150 items quickly become prohibitive for classical MPS-based simulation, which contrasts the relatively constant execution times of the same circuits on quantum hardware. Consequently, while larger instances beyond those considered in this work become classically intractable, the corresponding quantum circuits remain practically executable, provided sufficiently capable hardware is available.

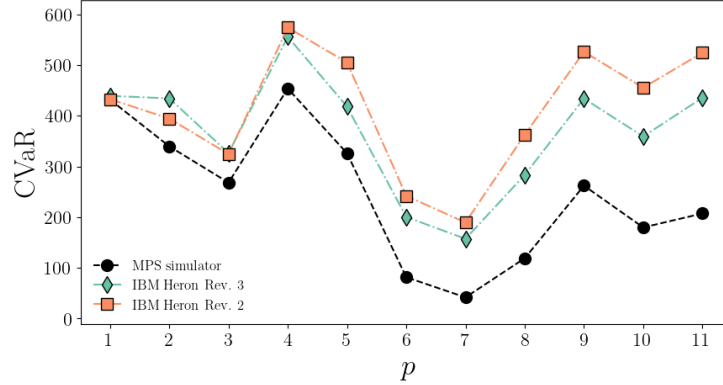


Figure 3: CVaR as a function of circuit depth p for a 50 item, 5 constraint knapsack instance.

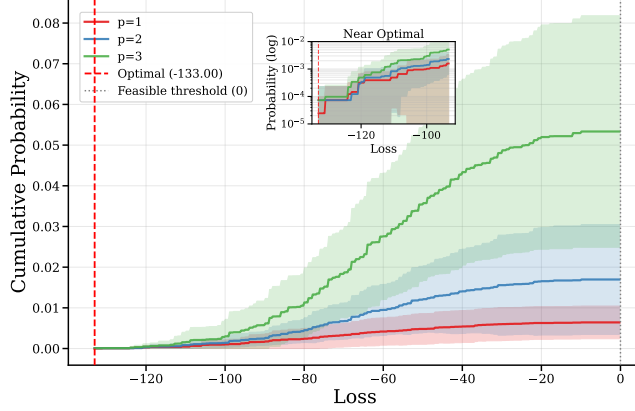


Figure 4: Empirical CDF of the loss L for circuit depths $p = 1, 2, 3$ on the 20 item, 5 constraint CCKP instance. The plot displays the mean CDF with shaded confidence regions (± 1 standard deviation over 10 independent runs of the variational scheme) for the feasible region ($L < 0$) under the chosen penalty λ . An inset with logarithmic y-axis focuses on the near-optimal region ($-140 \leq L \leq -93$), highlighting the probability of achieving solutions close to the optimum. The vertical dashed line at $L = -133$ marks the optimal solution, while the dotted line at $L = 0$ indicates the feasibility threshold.

To investigate hardware constraints, Fig. 3 compares the CVaR as a function of circuit depth p across the MPS simulator and two IBM Heron processors, Revision 2 and Revision 3, for a 50 item problem with 5 constraints. Overall, increasing p up to $p = 7$ improves CVaR, though the trend is non-monotonic, consistent with the preceeding analysis. Across all p , hardware implementations exhibit higher CVaR values than their noiseless simulation counterparts, as expected due to noise. Beyond $p > 7$, hardware noise begins to dominate, leading to unreliable CVaR values and, in some cases, degradation in the performance of the scheme.

Empirical cumulative distribution function. Fig. 4 depicts the empirical CDF constructed from the sampled solutions. Feasible samples are identified by negative loss values under Eq. (23), whereas samples with positive loss are infeasible. The samples were collected from a variational quantum circuit trained using MPS simulator corresponding to a 20 item problem with 5 constraints. The standard deviation was computed by averaging 10 independent runs of the variational scheme. As the circuit depth and corresponding circuit complexity increases, the likelihood of obtaining feasible solutions ($L < 0$) improves: for $p = 1$ the probability is $0.6\% \pm 0.4\%$, for $p = 2$ it is $1.7\% \pm 1.4\%$, and for $p = 3$ it reaches $5.4\% \pm 2.9\%$. Additionally, the inset in Fig. 4 shows that the optimal solution at loss $L = -133$ is sampled with measurement probability of $\sim 10^{-4}$ for $p = 3$.

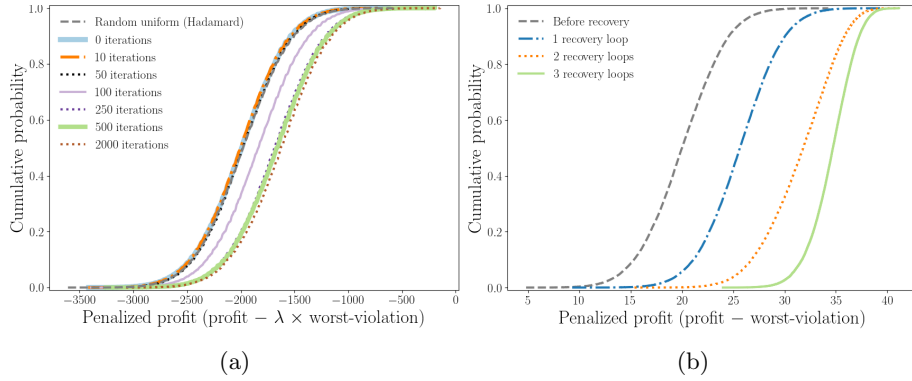


Figure 5: Empirical cumulative distributions of penalized profit for a 100-item, 5-constraint knapsack problem obtained from hardware-sampled quantum circuits. (a) Comparison of a random (Hadamard) baseline and increasing numbers of fine-tuning iterations following parameter transfer from a 50-item, 5-constraint knapsack. Profit is penalized using a λ -scaled worst-case SOCP constraint violation, where λ matches the value employed during CVaR-based training. Successive fine-tuning stages induce a progressive rightward shift of the distributions that saturates at 500 iterations (green), indicating increasing concentration of probability mass on higher-quality solutions under the training objective. (b) Empirical distributions of feasibility-aware profit before and after applying a fixed number of recovery-algorithm iterations. Here, profit is penalized by the unscaled worst-case SOCP violation, reflecting direct constraint enforcement independent of the CVaR training penalty.

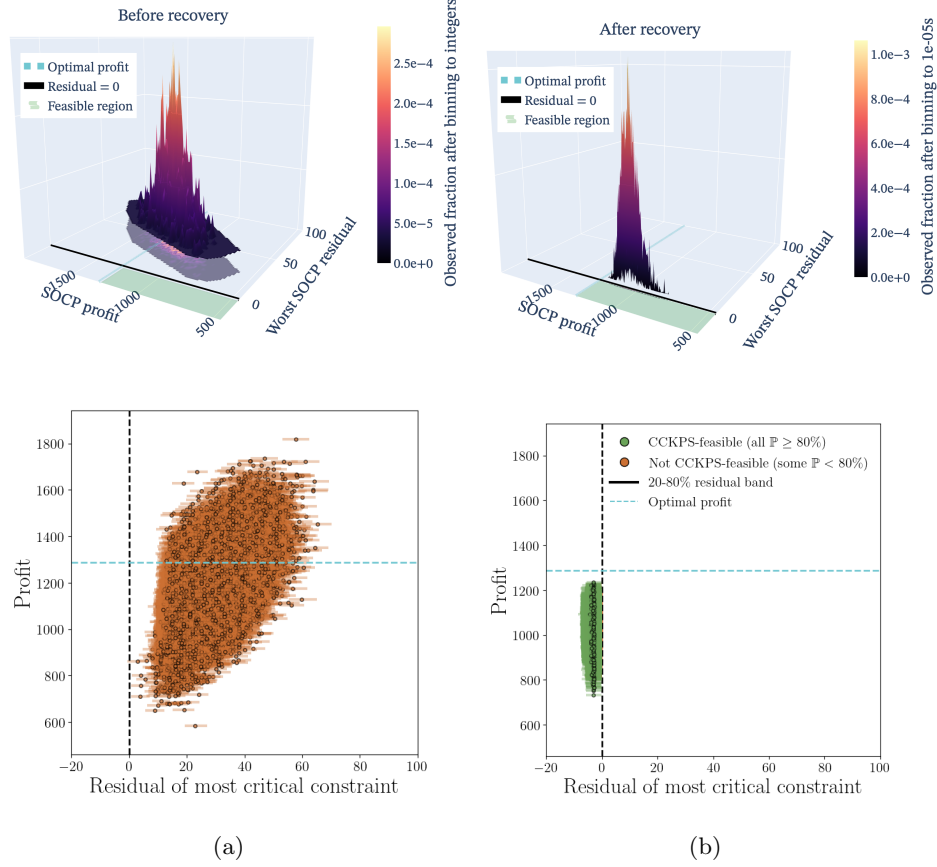


Figure 6: Visualizations of observed bitstrings for a 100-item, 5-constraint knapsack instance (a) before and (b) after recovery. Top: SOCP surface plots of bitstrings showing profit versus worst SOCP residual, with the colorbar representing observed hardware sampling probability. The blue line represents the classical best solution profit, and its intersection with the black line ($r^{(*)} = 0$) outlines the SOCP-feasible zone in the xy -plane (green rectangle). Bottom: CCKP scatter plots of bitstrings showing profit versus CCKP critical residuals with the highest violation probability. The dashed blue line once again indicates the classically best profit. Each point corresponds to a unique bitstring colored by CCKP-sampled (CCKPS) feasibility, and error bars denote the 20–80% range ($\epsilon = 0.2$) of sampled residuals $r^{(*)}$ from 1000 band samples. After recovery, nearly all bitstrings are CCKPS-feasible, approaching equivalence of CCKPS-feasibility to SOCP-feasibility under sufficient band samples.

Furthermore, Fig. 5 demonstrates how the loss function evolves across different optimization steps for circuits with 100 qubits, corresponding to a problem instance with 100 items and 5 constraints. Fig. 5a illustrates the evolution of the loss function during the circuit fine-tuning step, which is displayed as empirical CDFs of penalized profit. Samples were obtained from hardware after classical fine-tuning of varying maximum number of iterations⁵ with the MPS simulator. Specifically, the case of 0 iterations represents a circuit initialized by direct parameter transfer from a 50-qubit instance, whereas iteration counts greater than 0 reflect additional fine-tuning steps carried out using the MPS simulator. The calculation of the penalized profit was set by subtracting a λ -weighted worst-case SOCP constraint violation, with λ fixed to the value used during CVaR-based training to ensure consistency between the training objective and post-hoc evaluation.

The warm-start (0 iteration) empirical distribution initially performs similarly to a random uniform baseline, with its effectiveness depending sensitively on the choice of circuit depth p used for parameter transfer. Parameter transfer nonetheless provides a clear benefit by stabilizing and accelerating CVaR convergence, suggesting improved guidance of the optimization trajectory even when immediate gains in solution quality are limited. As shown in Fig. 5a, the fine-tuned distributions begin to outperform the random baseline after approximately 10 iterations, as indicated by a rightward shift in the CDF. However, for fewer than 100 fine-tuning iterations, these improvements remain relatively modest, partially because COBYLA expects to perform at least $P \cdot (2 + M) + 2$ function evaluations.

As additional fine-tuning stages are applied, the distributions exhibit a monotonic rightward shift, indicating a systematic reallocation of probability mass toward more profitable solutions. This improvement begins to plateau around 500 iterations, beyond which no further changes in the CDF are observed. The observed behavior reflects improved constraint satisfaction in the tail of the sampled distribution, together with increased alignment with the risk-averse training objective imposed by the CVaR loss.

Fig. 5b illustrates the evolution of feasibility-aware profit distributions under successive applications of the recovery algorithm to the 100 item, 5 constraint knapsack after fine-tuning iterations plateaued at 500 iterations. In this figure, penalized profit aligns with the recovery penalty, which, in contrast to Fig. 5a, does not incur a scaling coefficient to the worst-case SOCP violation. The initial distribution corresponds to solutions directly obtained from hardware sampling and contains nonzero mass violating constraints. With increasing recovery iterations, the violating mass progressively redistributes toward higher penalized profit values, indicating the iterative correction of infeasible bitstrings. The dominant transformation occurs after the first recovery loop, which induces the largest rightward shift in the distribution. Subsequent recovery, particularly at three loops, contributes to variance reduction, tightening the distribution while shifting its mean to a lesser extent. After three loops, all bitstrings are

⁵Using COBYLA, the term iteration here implies a function evaluation.

SOCP-feasible, terminating the recovery step.

SOCP and CCKP relationship. To compare feasibility assessed under the SOCP relaxation and under CCKP sampling, Fig. 6 displays visualizations of hardware-sampled results before and after the application of sample recovery on the 100 item, 5 constraint knapsack. The two surface plots depict SOCP-feasibility for bitstrings before and after applying the recovery algorithm and demonstrate how the initially broad distributions produced by hardware sampling systematically coalesces probability mass near the intersection of low constraint residuals and the classically determined optimal profit. Complementary, the two scatter plots assess feasibility under CCKP sampling (CCKPS), with sample weights drawn from the 20–80% range of the corresponding CCKP distribution with 1000 band samples. The band sampling range was selected as $[\epsilon, 1 - \epsilon]$ to match the SOCP formulation.

Taken together, these surface and scatter plots enable a direct comparison between SOCP-feasible and CCKPS-feasible solutions. After recovery, the majority of bitstrings agree on feasibility under both formulations, and as the number of CCKPS band samples increases, this agreement improves monotonically. This convergence indicates that dense CCKP sampling recovers the same feasible set identified by the SOCP relaxation, while the recovery procedure drives the distribution toward constraint-saturating, high-profit configurations near the SOCP/CCKPS-feasibility boundary.

4 Conclusions and Outlook

In this work, we introduced a novel quantum–classical framework for solving CCKPs, motivated by stochastic decision-making challenges that arise in insurance underwriting and related domains. By reformulating probabilistic constraints into deterministic equivalents and leveraging a QAOA-based variational quantum circuit, we demonstrated how current quantum devices can be used as effective heuristic samplers for stochastic optimization. The proposed scheme integrates variational training, parameter transfer across problem sizes, and problem-aware classical post-processing to overcome known challenges such as barren plateaus [15, 16] and hardware noise. Overall, empirical results show that the quantum-generated samples concentrate probability mass on feasible, high-quality solutions, while maintaining moderate circuit depths.

These findings indicate a promising regime in which our proposed quantum-classical scheme can potentially yield substantial gains over purely classical approaches. Although the method consistently produces higher-quality solutions, the fine-tuning stage currently requires a relatively high number of optimizer iterations. Beyond parameter transfer, alternative “offline” circuit-parameter training strategies may help reduce this cost and will be explored in future work. Moreover, the ability to address larger, more constrained problems is expected to widen the performance gap between classical and quantum solvers, motivating continued investigation of scalable solution methods.

Beyond the specific knapsack formulation studied here, the methods point toward a more general paradigm for applying quantum computing to real-world stochastic optimization problems. Combining a specific structure in quantum circuits, risk-aware objective formulations, and robust classical post-processing provides a flexible template that can be adapted to other chance-constrained and uncertainty-driven decision problems. Although current hardware limitations restrict problem sizes and achievable depths, the results presented here suggest a pathway for scaling as quantum devices and error-mitigation techniques improve. From an industry perspective, this work highlights how hybrid quantum–classical approaches can be meaningfully aligned with business-relevant risk–reward trade-offs, offering a promising direction for future research and experimentation at the intersection of quantum computing and enterprise optimization.

References

- [1] Vineet Goyal and R. Ravi. A PTAS for the chance-constrained knapsack problem with random item sizes. *Operations Research Letters*, 38(3):161–164, 2010.
- [2] Roberto Rossi, Steven D. Prestwich, and S. Armagan Tarim. Mixed-integer linear programming approximations for the stochastic knapsack, 2025.
- [3] Zuo Zeng, Brianna Christian, and Selen Cremaschi. A generalized Knapsack-Problem based decomposition heuristic for solving multistage stochastic programs with endogenous and/or exogenous uncertainties. *Ind. Eng. Chem. Res.*, 57(28):9185–9199, July 2018.
- [4] David P. Morton and R. Kevin Wood. *On a Stochastic Knapsack Problem and Generalizations*, pages 149–168. Springer US, Boston, MA, 1998.
- [5] Masoud Mohseni et al. Challenges and opportunities in quantum optimization. *Nature Reviews Physics*, 2024.
- [6] Monit Sharma, Yan Jin, Hoong Chuin Lau, and Rudy Raymond. Quantum relaxation for solving multiple knapsack problems. *arXiv preprint*, 2024. Also presented at IEEE Quantum Week (QCE) 2024.
- [7] Wojciech Bożejko, Anna Burduk, Jarosław Pempera, Mariusz Uchroński, and Mieczysław Wodecki. Optimal solving of a binary knapsack problem on a d-wave quantum machine and its implementation in production systems. *Annals of Operations Research*, 2024.
- [8] Dylan Herman et al. Quantum computing for finance. *Nature Reviews Physics*, 5:450–465, 2023.
- [9] Narendra N. Hegade et al. Enhancing knapsack-based financial portfolio optimization using quantum approximate optimization algorithm. *arXiv preprint arXiv:2402.07123*, 2024.

- [10] Wim van Dam, Karim Eldefrawy, Nicholas Genise, and Natalie Parham. Quantum optimization heuristics with an application to knapsack problems. *arXiv preprint*, 2021.
- [11] Sören Wilkening, Andreea-Iulia Lefterovici, Lennart Binkowski, Michael Perk, Sándor Fekete, and Tobias J. Osborne. A quantum algorithm for solving 0-1 knapsack problems. *npj Quantum Information*, 2025. arXiv:2310.06623.
- [12] Naeimeh Mohseni, Julien-Pierre Houle, Ibrahim Shehzad, Giorgio Cortiana, Corey O’Meara, and Adam Bene Watts. Constrained quantum optimization at utility scale: Application to the knapsack problem. *arXiv preprint*, 2026.
- [13] David Ribes and Tatiana Gonzalez Grandon. Towards quantum stochastic optimization for energy systems under uncertainty: Joint chance constraints with quantum annealing. *arXiv preprint arXiv:2512.03925*, 2025.
- [14] Einar Gabbassov, Gili Rosenberg, and Artur Scherer. Lagrangian duality in quantum optimization: Overcoming qubo limitations for constrained problems. *Phys. Rev. Res.*, 7:023305, Jun 2025.
- [15] Martín Larocca, Supanut Thanasilp, Samson Wang, Kunal Sharma, Jacob Biamonte, Patrick J. Coles, Lukasz Cincio, Jarrod R. McClean, Zoë Holmes, and M. Cerezo. Barren plateaus in variational quantum computing. *Nature Reviews Physics*, 7(4):174–189, March 2025.
- [16] Jarrod R McClean, Sergio Boixo, Vadim N Smelyanskiy, Ryan Babbush, and Hartmut Neven. Barren plateaus in quantum neural network training landscapes. *Nature communications*, 9(1):4812, 2018.
- [17] Gurobi Optimization, LLC. Gurobi Optimizer Reference Manual, 2026.
- [18] Edward Farhi, Jeffrey Goldstone, and Sam Gutmann. A quantum approximate optimization algorithm. *arXiv preprint arXiv:1411.4028*, 2014.
- [19] Panagiotis Kl. Barkoutsos, Giacomo Nannicini, Anton Robert, Ivano Tavernelli, and Stefan Woerner. Improving Variational Quantum Optimization using CVaR. *Quantum*, 4:256, April 2020.
- [20] M. J. D. Powell. *A Direct Search Optimization Method That Models the Objective and Constraint Functions by Linear Interpolation*, pages 51–67. Springer Netherlands, Dordrecht, 1994.
- [21] Pauli Virtanen, Ralf Gommers, Travis E. Oliphant, et al. SciPy 1.0: Fundamental algorithms for scientific computing in python. *Nature Methods*, 17:261–272, 2020.

- [22] Samantha V. Barron, Daniel J. Egger, Elijah Pelofske, Andreas Bärttschi, Stephan Eidenbenz, Matthis Lehmkuehler, and Stefan Woerner. Provable bounds for noise-free expectation values computed from noisy samples. *Nature Computational Science*, 4(11):865–875, November 2024.
- [23] Harold W. Kuhn. The Hungarian method for the assignment problem. *Naval Research Logistics Quarterly*, 2:83–97, 1955.
- [24] IBM Quantum. Qiskit runtime. <https://quantum.cloud.ibm.com/>, 2026. Online; accessed 1-April-2026.
- [25] Guifré Vidal. Efficient classical simulation of slightly entangled quantum computations. *Physical review letters*, 91(14):147902, 2003.
- [26] Qiskit aer. <https://github.com/Qiskit/qiskit-aer>. Online; accessed 1-April-2026.
- [27] Nic Ezzell, Bibek Pokharel, Lina Tewala, Gregory Quiroz, and Daniel A. Lidar. Dynamical decoupling for superconducting qubits: A performance survey. *Phys. Rev. Appl.*, 20:064027, Dec 2023.
- [28] Javier Robledo-Moreno, Mario Motta, Holger Haas, Ali Javadi-Abhari, Petar Jurcevic, William Kirby, Simon Martiel, Kunal Sharma, Sandeep Sharma, Tomonori Shirakawa, et al. Chemistry beyond the scale of exact diagonalization on a quantum-centric supercomputer. *Science Advances*, 11(13):eadu9991, 2025. arXiv:2405.05068 [quant-ph].
- [29] Guifré Vidal. Efficient classical simulation of slightly entangled quantum computations. *Physical Review Letters*, 91(14), October 2003.
- [30] Gabin Schieffer, Stefano Markidis, and Ivy Peng. Harnessing cuda-q’s mps for tensor network simulations of large-scale quantum circuits, 2025.

Appendix

A Derivation

We restate Theorem 1 for convenience.

Theorem 1. *For the mutually independent knapsack weights $\tilde{w}_i^{(m)}$ chosen from a distributions $\Delta(\mu, \sigma)$ closed under convolution with mean $\mu_i^{(m)}$ and standard deviation $\sigma_i^{(m)}$, and its cumulative distribution function Φ strictly increasing and invertible, the chance-constrained problem in (1) can be rewritten as*

$$\max_{\mathbf{x} \in \{0,1\}^n} \sum_{i=1}^n v_i x_i \quad (2)$$

$$\begin{aligned} s.t. \quad & \sum_{i=1}^n \mu_i^{(m)} x_i + \Phi^{-1}(1 - \epsilon^{(m)}) \sqrt{\sum_{i=1}^n \sigma_i^{(m)^2} x_i} \leq C^{(m)} \\ & \forall \quad m \in \{1, 2, \dots, M\} \end{aligned} \quad (3)$$

where Φ^{-1} is the inverse of the cumulative distribution function (CDF) of the distribution $\Delta(0, 1)$ and some small $\epsilon^{(m)} > 0$.⁶

Proof. Let the total weight of the selected items corresponding to a constraint m be $B^{(m)} = \sum_{i=1}^n \tilde{w}_i^{(m)} x_i$. Each term $\tilde{w}_i^{(m)} x_i$ is either 0 or $\tilde{w}_i^{(m)}$, and the random variables $\tilde{w}_i^{(m)}$ are independent. Since the distribution Δ is closed under convolution, the random variable $B^{(m)}$ also follows the distribution Δ . Because $x_i \in \{0, 1\}$, it follows that $x_i^2 = x_i$, and thus the variance of $B^{(m)}$ simplifies to $\text{Var}(B^{(m)}) = \sum_{i=1}^n \sigma_i^{(m)^2} x_i$. Consequently,

$$B^{(m)} \sim \Delta(\mu^{(m)}(\mathbf{x}), \sigma^{(m)^2}(\mathbf{x}))$$

where $\mu^{(m)}(\mathbf{x}) = \sum_{i=1}^n \mu_i^{(m)} x_i$, and $\sigma^{(m)}(\mathbf{x}) = \sqrt{\sum_{i=1}^n \sigma_i^{(m)^2} x_i}$.

We impose the chance constraint

$$\mathbb{P}(B^{(m)} \leq C^{(m)}) \geq 1 - \epsilon.$$

By standardizing $B^{(m)}$, we obtain

$$\frac{B^{(m)} - \mu^{(m)}(\mathbf{x})}{\sigma^{(m)}(\mathbf{x})} \sim \Delta(0, 1).$$

⁶For distributions satisfying the stated conditions, the probabilistic constraint in (1) admits an exact reformulation as the second-order cone constraint in (3). Classical solvers such as Gurobi handle these constraints by squaring both sides to obtain an equivalent quadratic constraint, enabling the use of second-order cone programming (SOCP) [1].

Hence

$$\begin{aligned}\mathbb{P}(B^{(m)} \leq C^{(m)}) &= \mathbb{P}\left(\frac{B^{(m)} - \mu^{(m)}(\mathbf{x})}{\sigma^{(m)}(\mathbf{x})} \leq \frac{C^{(m)} - \mu^{(m)}(\mathbf{x})}{\sigma^{(m)}(\mathbf{x})}\right) \\ &= \Phi\left(\frac{C^{(m)} - \mu^{(m)}(\mathbf{x})}{\sigma^{(m)}(\mathbf{x})}\right),\end{aligned}$$

where $\Phi(\cdot)$ denotes the cumulative distribution function (CDF) of the standard normal distribution. Additionally, we define

$$b := \frac{C^{(m)} - \mu^{(m)}(\mathbf{x})}{\sigma^{(m)}(\mathbf{x})}.$$

Then, the constraint $\Phi(b) \geq 1 - \epsilon$ implies

$$b \geq \Phi^{-1}(1 - \epsilon) \quad \Rightarrow \quad \frac{C^{(m)} - \mu^{(m)}(\mathbf{x})}{\sigma^{(m)}(\mathbf{x})} \geq \Phi^{-1}(1 - \epsilon),$$

where this inversion is valid because the CDF Φ is strictly increasing, which allows us to apply its inverse to both sides of the inequality $\Phi(b) \geq 1 - \epsilon$.

Assuming $\sigma(\mathbf{x}) > 0$ and $\Phi^{-1}(1 - \epsilon) > 0$ (equivalently, $1 - \epsilon > 0.5$), we can rearrange the inequality to obtain

$$C^{(m)} - \mu^{(m)}(\mathbf{x}) \geq \Phi^{-1}(1 - \epsilon) \sigma^{(m)}(\mathbf{x}),$$

that is,

$$\sum_{i=1}^n \mu_i^{(m)} x_i + \Phi^{-1}(1 - \epsilon) \sqrt{\sum_{i=1}^n \sigma_i^{(m)^2 x_i} \leq C^{(m)}}.$$

□

B Extension to the full XX chain

The two-state analysis Sect. 2.3.1 extends to the full mixer acting on all $\binom{n}{k^*}$ states in the feasible sector. We work within the Hamming-weight sector $\mathcal{H}_{k^*}$ and derive the *conditional* probability of measuring a given k^* -subset, given that the measurement outcome has Hamming weight k^* . In our scheme, the penalty term in the loss function concentrates amplitude in the feasible sector during optimization; the analysis below characterizes how the mixer redistributes that amplitude among feasible solutions.

State after the phase separator. Let $\mathcal{T} \subseteq \{1, \dots, n\}$ denote a k^* -element subset, and let $|\mathcal{T}\rangle$ denote the corresponding computational-basis state. Since the phase separator is diagonal with effective rotation $\phi_j = \alpha v_j - \beta \mu_j$ on qubit j , the eigenvalue associated with $\mathcal{T}$ is

$$E_{\mathcal{T}} = \sum_{j \notin \mathcal{T}} \phi_j - \sum_{j \in \mathcal{T}} \phi_j. \quad (31)$$

Starting from a uniform superposition over k^* -subsets,

$$|\psi_0\rangle = \sum_{\mathcal{T}} \omega_{\mathcal{T}} |\mathcal{T}\rangle, \quad \omega_{\mathcal{T}} = \binom{n}{k^*}^{-1/2},$$

the phase separator produces

$$|\psi_1\rangle = \sum_{\mathcal{T}} \omega_{\mathcal{T}} e^{-iE_{\mathcal{T}}} |\mathcal{T}\rangle. \quad (32)$$

Probabilities are unchanged: $|\langle \mathcal{T} | \psi_1 \rangle|^2 = |\omega_{\mathcal{T}}|^2$ for all $\mathcal{T}$.

First-order mixer expansion. The projected mixer within $\mathcal{H}_{k^*}$ is $H_{\text{mix}}^{(k^*)} = -\sum_{(j,j') \in E(G)} T_{jj'}$, where $T_{jj'}$ is the swap operator and $E(G)$ is the edge set of the mixer graph. Expanding the mixer unitary to first order in γ ,

$$|\psi_2\rangle = e^{-i\gamma H_{\text{mix}}^{(k^*)}} |\psi_1\rangle \approx |\psi_1\rangle + i\gamma \sum_{(j,j') \in E(G)} T_{jj'} |\psi_1\rangle. \quad (33)$$

Amplitude of a target subset $\mathcal{T}$. We compute $\langle \mathcal{T} | \psi_2 \rangle$ by evaluating $\langle \mathcal{T} | T_{jj'} | \psi_1 \rangle$ for each edge. For a fixed target $\mathcal{T}$ and edge (j, j') , the only source subset $\mathcal{T}'$ such that $T_{jj'} |\mathcal{T}'\rangle = |\mathcal{T}\rangle$ is $\mathcal{T}' = T_{jj'}(\mathcal{T})$. There are two cases:

1. **Non-boundary edge** (both $j, j' \in \mathcal{T}$ or both $j, j' \notin \mathcal{T}$): the swap acts as the identity on $|\mathcal{T}\rangle$, so $\mathcal{T}' = \mathcal{T}$ and

$$\langle \mathcal{T} | T_{jj'} | \psi_1 \rangle = \omega_{\mathcal{T}} e^{-iE_{\mathcal{T}}}. \quad (34)$$

2. **Boundary edge** ($j \in \mathcal{T}, j' \notin \mathcal{T}$): the swap maps $|\mathcal{T}'\rangle \rightarrow |\mathcal{T}\rangle$ with

$$\mathcal{T}' = \mathcal{T} \setminus \{j\} \cup \{j'\},$$

giving

$$\langle \mathcal{T} | T_{jj'} | \psi_1 \rangle = \omega_{\mathcal{T}'} e^{-iE_{\mathcal{T}'}}. \quad (35)$$

For a boundary edge, the eigenvalue difference follows from (31). In $\mathcal{T}$, item j contributes $-\phi_j$ and j' contributes $+\phi_{j'}$, while in $\mathcal{T}'$ these roles are reversed. Therefore,

$$E_{\mathcal{T}'} - E_{\mathcal{T}} = 2(\phi_j - \phi_{j'}). \quad (36)$$

Collecting all edges and using $\omega_{\mathcal{T}} = \omega_{\mathcal{T}'}$,

$$\langle \mathcal{T} | \psi_2 \rangle = \omega_{\mathcal{T}} e^{-iE_{\mathcal{T}}} \left[1 + i\gamma n_0(\mathcal{T}) + i\gamma \sum_{\substack{(j,j') \in E(G) \\ j \in \mathcal{T}, j' \notin \mathcal{T}}} e^{-2i(\phi_j - \phi_{j'})} \right], \quad (37)$$

where $n_0(\mathcal{T})$ is the number of non-boundary edges.

Measurement probability. Taking the modulus squared to first order in γ ,

$$\mathbb{P}(\mathcal{T}) = |\langle \mathcal{T} | \psi_2 \rangle|^2 \approx \omega_{\mathcal{T}}^2 \left(1 + 2 \operatorname{Re} [i\gamma n_0(\mathcal{T}) + i\gamma \sum e^{-2i(\phi_j - \phi_{j'})}] \right). \quad (38)$$

The non-boundary term is purely imaginary and does not contribute. For each boundary edge, using $\text{Re}(ie^{-i\theta}) = \sin \theta$,

$$\text{Re}\left(i\gamma e^{-2i(\phi_j - \phi_{j'})}\right) = \gamma \sin(2(\phi_j - \phi_{j'})). \quad (39)$$

Substituting $\phi_j = \alpha v_j - \beta \mu_j$ yields

$$\mathbb{P}(\mathcal{T}) \approx |\omega_{\mathcal{T}}|^2 \left(1 + 2\gamma \sum_{\substack{(j, j') \in E(G): \\ j \in \mathcal{T}, j' \notin \mathcal{T}}} \sin\left(2[\alpha(v_j - v_{j'}) - \beta(\mu_j - \mu_{j'})]\right) \right), \quad (40)$$

where $j \in \mathcal{T}$ denotes an included item and $j' \notin \mathcal{T}$ its excluded neighbor on the mixer graph.

C Post-Processing Algorithm

The post-processing scheme takes a set of binary samples as input. These samples can be potentially noisy outputs from a quantum device, and the recovery scheme iteratively refines them through three stages: (i) objective-based weighting, (ii) batched probability update, and (iii) two-phase sample repair. The overall procedure is self-consistent: repaired samples feed back into the probability estimates, which in turn guide the next round of repairs.

C.1 Objective-Based Weighting

Feasible samples are assigned exponential (softmax) weights proportional to their profit, so that better-quality solutions have more weight during the probability update step.

Algorithm 1 Compute Objective-Based Weights

Input: Feasible samples $\mathcal{S}_{\text{feas}}$, profit function $f(\mathbf{x}) = \mathbf{v}^\top \mathbf{x}$

Output: Weight vector $\mathbf{w}$

- 1: $f_{\max} \leftarrow \max_{\mathbf{x} \in \mathcal{S}_{\text{feas}}} f(\mathbf{x})$
 - 2: **for** each $\mathbf{x}_k \in \mathcal{S}_{\text{feas}}$ **do**
 - 3: $w_k \leftarrow \frac{\exp(f(\mathbf{x}_k) - f_{\max})}{\sum_{k'} \exp(f(\mathbf{x}_{k'}) - f_{\max})}$
 - 4: **end for**
 - 5: **return** $\mathbf{w}$
-

Subtracting $f_{\max}$ in the exponent ensures numerical stability, ensuring all exponents are negative.

C.2 Batched Probability Update

Bit-wise selection probabilities are updated via a batched weighted average. Partitioning into K batches and averaging the per-batch estimates provides variance reduction compared to a single global average.

Algorithm 2 Update Selection Probabilities

Input: $\mathcal{S}_{\text{feas}}$, weights $\mathbf{w}$, number of batches K , number of items n

Output: Updated probabilities $\mathbf{p}^{\text{new}} \in [0, 1]^n$

1: Partition $\mathcal{S}_{\text{feas}}$ into K batches $\mathcal{B}_1, \dots, \mathcal{B}_K$

2: **for** $i = 1$ to n **do**

3: $p_i^{\text{new}} \leftarrow \frac{1}{K} \sum_{\ell=1}^K \frac{\sum_{\mathbf{x} \in \mathcal{B}_\ell} w_{\mathbf{x}} x_i}{\sum_{\mathbf{x} \in \mathcal{B}_\ell} w_{\mathbf{x}}}$

4: **end for**

5: **return** $\mathbf{p}^{\text{new}}$

C.3 Two-Phase Sample Repair

Each sample undergoes a two-phase greedy repair procedure to increase feasibility and objective value among samples.

Algorithm 3 Two-Phase Sample Repair

Input: Sample $\mathbf{x}$, probabilities $\mathbf{p}$, values $\mathbf{v}$, constraint data $(W, \boldsymbol{\sigma}, \mathbf{C}, \Phi^{-1})$, flip budgets

$F_{\text{rem}}, F_{\text{add}}$

Output: Repaired sample $\mathbf{x}'$

```

1:  $\mathbf{x}' \leftarrow \mathbf{x}$ 
2:
3: — Phase 1: Feasibility Repair —
4: for  $t = 1$  to  $F_{\text{rem}}$  do
5:   if  $\mathbf{x}'$  is feasible then
6:     break
7:   end if
8:    $V_{\text{tot}} \leftarrow \sum_{m=1}^M g^{(m)}(\mathbf{x}')$  ▷ total violation
9:    $i^* \leftarrow -1; s^* \leftarrow -\infty$ 
10:  for each  $i$  with  $x'_i = 1$  do
11:     $\mathbf{x}^{\text{test}} \leftarrow \mathbf{x}'$  with  $x_i^{\text{test}} \leftarrow 0$ 
12:     $\text{relief}_i \leftarrow V_{\text{tot}} - \sum_m g^{(m)}(\mathbf{x}^{\text{test}})$ 
13:    if  $\text{relief}_i \leq 0$  then
14:      continue
15:    end if
16:     $\text{score}_i \leftarrow \frac{\text{relief}_i}{\max(v_i, \delta)} + \eta(1 - p_i)$  ▷ prefer low-prob, low-value items
17:    if  $\text{score}_i > s^*$  then
18:       $i^* \leftarrow i; s^* \leftarrow \text{score}_i$ 
19:    end if
20:  end for
21:  if  $i^* \geq 0$  then
22:     $x'_{i^*} \leftarrow 0$ 
23:  else
24:    break ▷ no improving removal; stop
25:  end if
26: end for
27:
28: — Phase 2: Greedy Improvement —
29: if  $\mathbf{x}'$  is feasible then
30:   Sort unselected items by  $v_i$  descending
31:    $t \leftarrow 0$ 
32:   for each unselected item  $i$  in sorted order do
33:     if  $t \geq F_{\text{add}}$  then
34:       break
35:     end if
36:      $\mathbf{x}^{\text{test}} \leftarrow \mathbf{x}'$  with  $x_i^{\text{test}} \leftarrow 1$ 
37:     if  $\mathbf{x}^{\text{test}}$  is feasible then
38:        $x'_i \leftarrow 1; t \leftarrow t + 1$ 
39:     end if
40:   end for
41: end if
42: return  $\mathbf{x}'$ 

```

Phase 1 focuses exclusively on eliminating constraint violations by removing items. The scoring function

$$\text{score}_i = \frac{\text{relief}_i}{\max(v_i, \delta)} + \eta(1 - p_i)$$

balances two considerations: (a) maximizing constraint relief per unit of lost profit, and (b) preferring to remove items with low selection probability p_i ,

reflecting the collective signal from previously observed high-quality samples. Here, $\delta > 0$ is a small constant to avoid division by zero and $\eta \geq 0$ controls the strength of the probability bias.

Phase 2 then exploits any remaining capacity slack by greedily adding the highest-value items that fit within the constraints. The SOCP constraint (3) is non-separable since the variance term couples all selected items, meaning that each candidate addition must be checked against the full constraint set.

In the case where no single removal reduces the total violation, Phase 1 cannot fully restore feasibility. Therefore, the sample is returned as-is rather than replaced by a random draw from $\mathbf{p}$. This preserves structural information from the original quantum sample; infeasible samples are subsequently filtered out during the probability update step.

C.4 Main Self-Consistent Loop

The three components are combined in an iterative loop, shown below, that alternates between repairing samples and updating the probability model until convergence.

Algorithm 4 Self-Consistent Probability-Based Recovery

Input: Initial samples $\mathcal{S}_0$, profit function f , batches K , tolerance ϵ_{conv} , max iterations $\mathcal{T}$

Output: Best feasible solution $\mathbf{x}^*$

- 1: Partition $\mathcal{S}_0 \rightarrow (\mathcal{S}_{\text{feas}}, \mathcal{S}_{\text{infeas}})$ by constraint satisfaction
 - 2: Initialize $\mathbf{p} \in [0, 1]^n$ from $\mathcal{S}_{\text{feas}}$
 - 3: $t \leftarrow 0$
 - 4: **repeat**
 - 5: **Repair:** Apply Algorithm 3 to every sample in $\mathcal{S}_{\text{feas}} \cup \mathcal{S}_{\text{infeas}}$
 - 6: Collect all repaired samples into $\mathcal{S}_{\text{rec}}$
 - 7: Filter: $\mathcal{S}_{\text{feas}}^{\text{rec}} \leftarrow \{\mathbf{x} \in \mathcal{S}_{\text{rec}} : r^{(m)}(\mathbf{x}) \leq 0 \forall m\}$
 - 8: **if** $\mathcal{S}_{\text{feas}}^{\text{rec}} = \emptyset$ **then**
 - 9: $\mathcal{S}_{\text{feas}}^{\text{rec}} \leftarrow \mathcal{S}_{\text{rec}}$ ▷ use all if none feasible
 - 10: **end if**
 - 11: $\mathbf{w} \leftarrow \text{COMPUTEWEIGHTS}(\mathcal{S}_{\text{feas}}^{\text{rec}}, f)$ ▷ Algorithm 1
 - 12: $\mathbf{p}^{\text{new}} \leftarrow \text{UPDATEPROBABILITIES}(\mathcal{S}_{\text{feas}}^{\text{rec}}, \mathbf{w}, K, n)$ ▷ Algorithm 2
 - 13: $\Delta \leftarrow \frac{1}{n} \sum_{i=1}^n |p_i^{\text{new}} - p_i|$
 - 14: Re-partition $\mathcal{S}_{\text{rec}}$ into $\mathcal{S}_{\text{feas}}$ and $\mathcal{S}_{\text{infeas}}$ by constraint satisfaction
 - 15: $\mathbf{p} \leftarrow \mathbf{p}^{\text{new}}; \quad t \leftarrow t + 1$
 - 16: **until** $\Delta < \epsilon_{\text{conv}} \vee t \geq \mathcal{T}$
 - 17: **return** $\mathbf{x}^* \leftarrow \arg \max_{\mathbf{x} \in \mathcal{S}_{\text{feas}}} f(\mathbf{x})$
-

Convergence. The loop terminates when the mean absolute change in selection probabilities falls below a threshold ϵ_{conv} , or after a maximum of $\mathcal{T}$ iterations. The repair step drives infeasible samples toward feasibility, the weighting

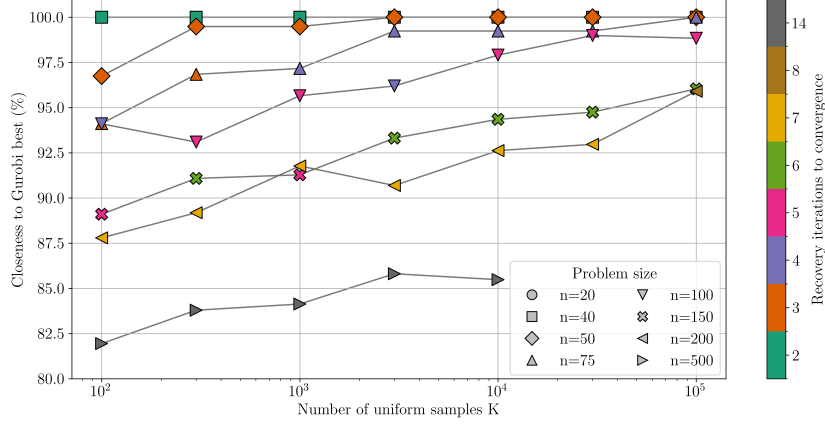


Figure 7: Effectiveness of the unconstrained recovery algorithm on K -randomly-sampled bitstrings from a uniform distribution obtained via Hadamard initialization on IBM Heron processors for $n \leq 150$ and using classical sampling for $n = 200$ and 500 . Marker shape indicates the problem size n , and marker color denotes the number of recovery iterations to convergence. As problem size increases, the recovered solutions tend to be farther from the Gurobi-best solution and require more recovery steps. Increasing the number of K -random samples tends to improve solution quality, reducing the closeness gap. At $n = 500$, prohibitive runtimes necessitated a cap of 14 recovery iterations, and results with $N_{\text{shots}} > 10^4$ are omitted.

step concentrates probability mass on high-profit regions, and the two mechanisms reinforce each other.

Computational cost. The recovery procedure runtime is primarily dominated by feasibility checks and the number of distinct bitstrings processed. Each feasibility evaluation requires matrix operations over $M \times n$ problem parameters and therefore costs $\mathcal{O}(Mn)$. For each recovery iteration, the algorithm processes a total of $\mathcal{S}$ unique bitstrings and applies the recovery routine to each of them. The routine performs up to F_{rem} specified feasibility-restoration steps, where each step evaluates the current bitstring together with up to F_{add} specified candidate flips. This yields at most $((F_{\text{rem}} + F_{\text{add}})n)$ feasibility evaluations per repaired sample, and the resulting worst-case per-iteration runtime scales as

$$\mathcal{O}\left(|\mathcal{S}|(F_{\text{rem}} + F_{\text{add}})Mn\right) \quad (41)$$

When F_{rem} , F_{add} , and M are treated as constants, this reduces to $\mathcal{O}(|\mathcal{S}|n)$, which is linear in both the number of processed bitstrings and problem size.

In practice, finite sampling budgets further reduce the scaling to be linear in n , and Fig. 7 details the dependence of the recovery procedure on sampling

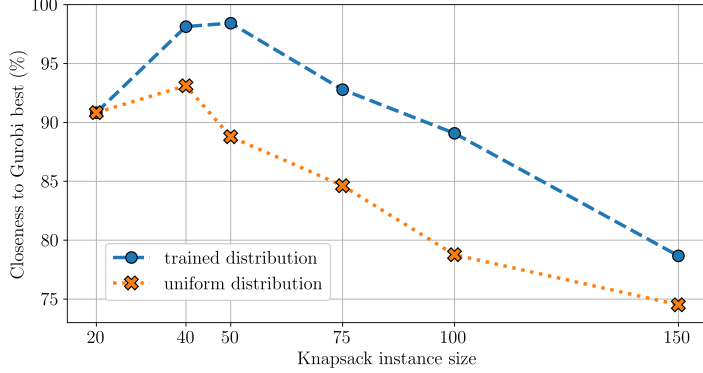


Figure 8: Closeness to the best Gurobi solution for knapsack instances of varying sizes under a recovery scheme with a limited removal budget equal to 10% of the instance size. Recovery initialized from a trained quantum circuit (blue) consistently yields solutions closer to the Gurobi best than recovery initialized from a uniform distribution (orange). The performance gap remains relatively constant as the knapsack size increases, indicating that the trained distribution provides a higher-quality starting point for scalable instances when under recovery budget constraints.

resources. An ablation study was performed in which inputs to the recovery scheme were samples drawn from a random uniform distribution. The total number of samples was varied between 10^2 and 10^5 , obtained either classically for $n > 150$ or from Hadamard-initialized quantum states on IBM Heron processors for $n \leq 150$. For each sample budget and problem size with $n < 500$, the recovery algorithm was executed until convergence, and both the number of recovery iterations required and the quality of the recovered solution relative to the Gurobi-best solution were recorded.

As expected, reducing the number of samples significantly decreases the probability of observing high-quality bitstrings near the Gurobi-best solution in the raw measurement data. This effect becomes increasingly pronounced as the problem size grows: the number of configurations expands combinatorially, while the probability mass associated with constraint-saturating, high-profit solutions correspondingly concentrates on a vanishingly small fraction of the state space. With fewer shots, such configurations are therefore sampled more sparsely or may be entirely absent, requiring additional recovery iterations to reconstruct them through local repairs.

Consistent with this interpretation, larger problem instances and lower shot budgets exhibit a marked increase in the number of recovery steps required to reach convergence, along with a degradation in the best recovered objective value relative to the Gurobi-best. This is particularly pronounced when $n = 500$, where excessive runtimes led to instantiating a cap on the number of recovery iterations to 14 and omission of results with $N_{\text{shots}} > 10^4$. Conversely,

increasing the shot budget improves sampling diversity, raising the likelihood that near-optimal bitstrings are present in the initial pool and accelerating convergence of the recovery procedure. Taken together, these results isolate the role of finite-shot sampling noise in governing recovery performance, disentangling sampling-driven effects from problem-size-dependent computational hardness.

In addition, the performance of the recovery algorithm depends upon the sampling distribution upon which it acts. Fig. 8 examines the effectiveness of the recovery scheme constrained to a limited removal flip budget, where the quality of the initial distribution becomes the dominant factor governing final performance. In this regime, recovery initialized from samples obtained via the quantum algorithm and optimized circuit consistently produce solutions closer to the Gurobi best across all instance sizes when compared to uniform distribution starting points. This behavior reflects the fact that a restricted recovery budget cannot compensate for poor initial samples. In particular, when starting from a uniform distribution, the sampled bitstrings are typically far from high-quality, constraint-satisfying configurations, requiring large coordinated changes that exceed the allowed number of flips. In contrast, the trained circuit distribution concentrates probability mass closer to near-feasible regions of the solution space, so that only small local repairs are needed to reach high-quality solutions. As the instance size grows, this distinction remains relatively constant, highlighting that under realistic, budget-limited recovery, improvements in the learned distribution translate directly into improved end-to-end solution quality.

D Classical solvers

D.1 Problem Formulation

All classical heuristic solvers use the deterministic form of the objective function [1]. For each constraint m , the left-hand side is computed as:

$$L_m(\mathbf{x}) = \sum_{i=1}^n \mu_i^{(m)} x_i + \Phi^{-1}(1 - \epsilon) \sqrt{\sum_{i=1}^n (\sigma_i^{(m)})^2 x_i} \quad (42)$$

The constraint violation is defined as $V_m(\mathbf{x}) = \max(0, L_m(\mathbf{x}) - C^{(m)})$, and the total violation across all constraints is $V(\mathbf{x}) = \sum_{m=1}^M V_m(\mathbf{x})$. The energy function to be maximized is then:

$$E(\mathbf{x}) = \mathbf{v}^\top \mathbf{x} - \lambda \cdot V(\mathbf{x}), \quad (43)$$

where $\lambda = \sum_{i=1}^n v_i$ is the penalty multiplier, chosen as the sum of all item values to provide a principled balance between objective maximization and constraint satisfaction. A solution is considered feasible if $L_m(\mathbf{x}) \leq C^{(m)}$ for all constraints $m \in \{1, \dots, M\}$. Additionally, throughout this section, $\mathbf{x}^*$ denotes the best feasible solution encountered so far; infeasible solutions may guide the search via the energy function but are not reported as final solutions.

All metaheuristic solvers in the following sections initialize solutions uniformly at random, with each binary variable x_i drawn independently from a Bernoulli distribution with probability 0.5. This ensures unbiased starting points across different runs and problem instances. For Parallel Tempering, each of the K replicas is initialized independently. For the Genetic Algorithm, all P individuals in the initial population are generated independently using this random initialization scheme.

All metaheuristics are subject to a wall-clock limit of 1800 seconds per instance. In addition, parallel tempering and simulated annealing are capped at 200,000 iterations, and the genetic algorithm at 2,000 generations with a population of 100 (i.e., 200,000 offspring evaluations). Tabu search differs in that each iteration evaluates up to 100 candidate neighbors rather than a single proposal; its cap is therefore set to 32,000 iterations, chosen such that its total number of loss-function evaluations ($\approx 3.2 \times 10^6$) matches that of parallel tempering, whose 8 replicas each evaluate two solutions per iteration.

D.2 Gurobi (Exact Solver)

See Sect. 2.2 for more about Gurobi.

D.3 Parallel Tempering

Parallel Tempering (PT) maintains multiple replicas of the solution at different temperatures, allowing efficient exploration of the solution space through both local moves and temperature swaps.

D.3.1 Implementation Details

Our implementation uses $K = 8$ temperature replicas with temperatures geometrically spaced between $T_{\min} = 0.1$ and $T_{\max} = 10.0$. Each replica performs Metropolis-Hastings steps where a single random bit is flipped to generate a neighbor solution. The acceptance probability follows the standard Metropolis criterion: moves that increase energy are always accepted, while moves that decrease energy are accepted with probability $\exp(\Delta E/T_k)$ where ΔE is the energy difference and T_k is the replica’s temperature. Every $S = 100$ iterations, adjacent replicas attempt to swap configurations using the parallel tempering acceptance criterion, which depends on both the energy difference and temperature difference between replicas.

D.4 Tabu Search

Tabu Search (TS) uses a memory structure to avoid cycling and encourage exploration of new regions of the solution space.

Algorithm 5 Parallel Tempering for Chance-Constrained knapsack

Input: n items, M constraints, parameters $\mathbf{v}, \boldsymbol{\mu}, \boldsymbol{\sigma}, \mathbf{C}, \epsilon$

Input: K replicas, temperatures $T_1 < T_2 < \dots < T_K$, swap interval S

```
1: Initialize  $K$  random solutions  $\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(K)}$ 
2:  $\mathbf{x}^* \leftarrow \arg \max_k E(\mathbf{x}^{(k)})$  ▷ Best solution
3: for  $t = 1$  to  $T_{\max}$  or until time limit do
4:   for  $k = 1$  to  $K$  do ▷ Metropolis step for each replica
5:      $\mathbf{x}' \leftarrow \mathbf{x}^{(k)}$ 
6:     Flip random bit:  $x'_i \leftarrow 1 - x_i$  for random  $i$ 
7:      $\Delta E \leftarrow E(\mathbf{x}') - E(\mathbf{x}^{(k)})$ 
8:     if  $\Delta E > 0$  or  $\text{rand}() < \exp(\Delta E/T_k)$  then
9:        $\mathbf{x}^{(k)} \leftarrow \mathbf{x}'$  ▷ Accept move
10:    end if
11:    if  $E(\mathbf{x}^{(k)}) > E(\mathbf{x}^*)$  and  $\mathbf{x}^{(k)}$  is feasible then
12:       $\mathbf{x}^* \leftarrow \mathbf{x}^{(k)}$ 
13:    end if
14:  end for
15:  if  $t \bmod S = 0$  then ▷ Attempt replica swaps
16:    for  $k = 1$  to  $K - 1$  do
17:       $\Delta \leftarrow (E(\mathbf{x}^{(k)}) - E(\mathbf{x}^{(k+1)})) \cdot (1/T_k - 1/T_{k+1})$ 
18:      if  $\Delta > 0$  or  $\text{rand}() < \exp(\Delta)$  then
19:        Swap  $\mathbf{x}^{(k)} \leftrightarrow \mathbf{x}^{(k+1)}$ 
20:      end if
21:    end for
22:  end if
23: end for
24: return  $\mathbf{x}^*$ 
```

Algorithm 6 Tabu Search for Chance-Constrained knapsack

Input: n items, M constraints, parameters $\mathbf{v}, \boldsymbol{\mu}, \boldsymbol{\sigma}, \mathbf{C}, \epsilon$

Input: Tabu tenure τ , max neighbors $N_{\max}$

```
1: Initialize random solution  $\mathbf{x}$ 
2:  $\mathbf{x}^* \leftarrow \mathbf{x}$  ▷ Best solution
3:  $\mathcal{T} \leftarrow \emptyset$  ▷ Tabu list (FIFO queue)
4: for  $t = 1$  to  $T_{\max}$  or until time limit do
5:    $\mathcal{N} \leftarrow \{\mathbf{x}' : \mathbf{x}' \text{ differs from } \mathbf{x} \text{ in one bit}\}$ 
6:   if  $|\mathcal{N}| > N_{\max}$  then
7:     Sample  $N_{\max}$  neighbors randomly from  $\mathcal{N}$ 
8:   end if
9:    $\mathbf{x}_{\text{best}} \leftarrow \arg \max_{\mathbf{x}' \in \mathcal{N}} E(\mathbf{x}')$ 
10:   $i_{\text{flip}} \leftarrow \text{index of flipped bit in } \mathbf{x}_{\text{best}}$ 
11:  if  $i_{\text{flip}} \notin \mathcal{T}$  or  $E(\mathbf{x}_{\text{best}}) > E(\mathbf{x}^*)$  then ▷ Aspiration
12:     $\mathbf{x} \leftarrow \mathbf{x}_{\text{best}}$ 
13:    Add  $i_{\text{flip}}$  to  $\mathcal{T}$  (remove oldest if  $|\mathcal{T}| = \tau$ )
14:    if  $E(\mathbf{x}) > E(\mathbf{x}^*)$  and  $\mathbf{x}$  is feasible then
15:       $\mathbf{x}^* \leftarrow \mathbf{x}$ 
16:    end if
17:  end if
18: end for
19: return  $\mathbf{x}^*$ 
```

D.4.1 Implementation Details

Our Tabu Search implementation maintains a tabu list with tenure $\tau = 20$ iterations, implemented as a first-in-first-out (FIFO) queue that stores the indices of recently flipped bits. At each iteration, we generate the neighborhood by considering single-bit flips if $n \leq N_{\max}$ from the current solution. When the neighborhood size exceeds $N_{\max} = 100$, we randomly sample $N_{\max}$ neighbors to maintain computational efficiency. The best neighbor is selected based on the energy function $E(\mathbf{x})$, and the corresponding bit flip is executed unless it is tabu. However, we employ an aspiration criterion that overrides the tabu status if the move leads to a solution better than the current best known solution.

D.5 Simulated Annealing

Simulated Annealing (SA) uses a temperature-based acceptance criterion with geometric cooling to gradually reduce exploration and focus on exploitation.

D.5.1 Implementation Details

Our Simulated Annealing implementation begins with an initial temperature of $T_0 = 10.0$ and cools geometrically with rate $\alpha = 0.95$ until reaching a final temperature of $T_f = 0.01$. At each temperature level, we perform $I = 1000$

Algorithm 7 Simulated Annealing for Chance-Constrained knapsack

Input: n items, M constraints, parameters $\mathbf{v}, \boldsymbol{\mu}, \boldsymbol{\sigma}, \mathbf{C}, \epsilon$

Input: Initial temp T_0 , final temp T_f , cooling rate α , iterations per temp I

```
1: Initialize random solution  $\mathbf{x}$ 
2:  $\mathbf{x}^* \leftarrow \mathbf{x}$  ▷ Best solution
3:  $\mathcal{T} \leftarrow T_0$ 
4: while  $\mathcal{T} > T_f$  and time limit not exceeded do
5:   for  $i = 1$  to  $I$  do
6:      $\mathbf{x}' \leftarrow \mathbf{x}$ 
7:     Flip random bit:  $x'_i \leftarrow 1 - x_i$  for random  $i$ 
8:      $\Delta E \leftarrow E(\mathbf{x}') - E(\mathbf{x})$ 
9:     if  $\Delta E > 0$  or  $\text{rand}() < \exp(\Delta E / \mathcal{T})$  then
10:       $\mathbf{x} \leftarrow \mathbf{x}'$  ▷ Accept move
11:    end if
12:    if  $E(\mathbf{x}) > E(\mathbf{x}^*)$  and  $\mathbf{x}$  is feasible then
13:       $\mathbf{x}^* \leftarrow \mathbf{x}$ 
14:    end if
15:  end for
16:   $\mathcal{T} \leftarrow \alpha \cdot \mathcal{T}$  ▷ Geometric cooling
17: end while
18: return  $\mathbf{x}^*$ 
```

iterations where a single random bit is flipped to generate a neighbor solution. The acceptance criterion follows the Metropolis rule: energy-increasing moves are always accepted, while energy-decreasing moves are accepted with probability $\exp(\Delta E / \mathcal{T})$ where ΔE is the energy change and $\mathcal{T}$ is the current temperature. After completing all iterations at a given temperature, we update the temperature as $T_{\text{new}} = \alpha \cdot T_{\text{old}}$.

D.6 Genetic Algorithm

The Genetic Algorithm (GA) maintains a population of solutions and evolves them using selection, crossover, and mutation operators inspired by natural evolution.

D.6.1 Genetic Operators

Tournament selection chooses a parent by randomly selecting k individuals from the population and returning the one with the highest fitness. Uniform crossover creates two offspring from two parents by independently choosing each bit position from either parent with equal probability, occurring with probability p_c . Bit-flip mutation independently flips each bit with probability p_m , introducing random variation to maintain population diversity.

Algorithm 8 Genetic Algorithm for Chance-Constrained knapsack

Input: n items, M constraints, parameters $\mathbf{v}, \boldsymbol{\mu}, \boldsymbol{\sigma}, \mathbf{C}, \epsilon$

Input: Population size P , crossover rate p_c , mutation rate p_m , tournament size k , elitism e

```
1: Initialize population  $\mathcal{P} = \{\mathbf{x}_1, \dots, \mathbf{x}_P\}$  randomly
2:  $\mathbf{x}^* \leftarrow \arg \max_{\mathbf{x} \in \mathcal{P}} E(\mathbf{x})$  ▷ Best solution
3: for  $g = 1$  to  $G_{\max}$  or until time limit do
4:   Evaluate fitness:  $f_i \leftarrow E(\mathbf{x}_i)$  for all  $\mathbf{x}_i \in \mathcal{P}$ 
5:   Sort  $\mathcal{P}$  by fitness (descending)
6:    $\mathcal{P}_{\text{new}} \leftarrow \{\mathbf{x}_1, \dots, \mathbf{x}_e\}$  ▷ Elitism
7:   while  $|\mathcal{P}_{\text{new}}| < P$  do
8:      $\mathbf{p}_1 \leftarrow \text{TournamentSelection}(\mathcal{P}, k)$ 
9:      $\mathbf{p}_2 \leftarrow \text{TournamentSelection}(\mathcal{P}, k)$ 
10:    if  $\text{rand}() < p_c$  then
11:       $(\mathbf{c}_1, \mathbf{c}_2) \leftarrow \text{UniformCrossover}(\mathbf{p}_1, \mathbf{p}_2)$ 
12:    else
13:       $(\mathbf{c}_1, \mathbf{c}_2) \leftarrow (\mathbf{p}_1, \mathbf{p}_2)$ 
14:    end if
15:     $\mathbf{c}_1 \leftarrow \text{Mutate}(\mathbf{c}_1, p_m)$ 
16:     $\mathbf{c}_2 \leftarrow \text{Mutate}(\mathbf{c}_2, p_m)$ 
17:    Add  $\mathbf{c}_1, \mathbf{c}_2$  to  $\mathcal{P}_{\text{new}}$ 
18:  end while
19:   $\mathcal{P} \leftarrow \mathcal{P}_{\text{new}}$ 
20:  if  $\max_{\mathbf{x} \in \mathcal{P}} E(\mathbf{x}) > E(\mathbf{x}^*)$  and feasible then
21:     $\mathbf{x}^* \leftarrow \arg \max_{\mathbf{x} \in \mathcal{P}} E(\mathbf{x})$ 
22:  end if
23: end for
24: return  $\mathbf{x}^*$ 
```

D.6.2 Implementation Details

Our Genetic Algorithm maintains a population of $P = 100$ individuals, each representing a binary solution vector. At each generation, we first evaluate the fitness of all individuals using the energy function $E(\mathbf{x})$. We preserve the $e = 2$ best individuals through elitism, directly copying them to the next generation. The remaining population is generated through tournament selection with tournament size $k = 3$, followed by uniform crossover with probability $p_c = 0.8$ and bit-flip mutation with per-bit probability $p_m = 0.01$.